

P1787R4: Declarations and where to find them

Audience: CWG

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

February 28, 2020

History

r4:

- Rebased onto N4849
- Forbid `decltype(...):~...`
- Restored block scope for most compound statements
- Fixed scope of parameters of a function declaration
- Fixed list of declarations and scope introducers
- Removed “direct function lvalue” interpretation for a *template-id*
- Made scope-based rules for copy elision and implicit moves consistent
- Removed truly final “point of declaration” usage
- Clarified terminology (in particular, for enclosing scopes)
- Added forward reference note for unusual declaration targets/binding
- Removed or labeled deprecated `::template` in examples
- Labeled `[class.mfct.non-static]` edits correctly

r3:

- Rebased onto N4835
- Added definition for “reachable from”
- Restored Richard’s rule for cross-TU *using-declaration* conflicts
- Generalized consistent-linkage rule for CWG1839
- Corrected scope of function-local predefined variables
- Updated all references to removed `[basic.lookup.classref]`, resolving CWG255
- Removed (remainder of) `[class.this]`
- Removed “global object” term

r2:

- Rebased onto N4830 (significant for using enumerators/enumerations)
- Added a locus for a *concept-definition*
- Extended containing scopes into redeclarations of classes and enumerations
- Properly limited cross-TU qualified lookup in namespaces
- Fixed rules for conflicts between *using-declarations*
- Renamed “anonymous union object” to “anonymous union variable”

r1:

- Fixed reversed *using-directive* containment test
- Updated **signature** definitions
- Removed residual “point of declaration” usage
- Fixed several uses of (new) “precede” to be (existing) “reachable”
- Removed dangling “considered to refer to a template” reference
- Used “be attached” for modules instead of (new, unrelated) “belong”
- Included class templates with classes for all scope purposes
- Distinguished entities from their associated scopes
- Accounted for missing kinds of declarations
- Removed superfluous definition of “denotes”
- Disentangled “enclose” and “contain” definitions
- Specified no lookup for `x` in `using x=...;`
- Allowed a *namespace-name* to be a member-qualified name
- Allowed in-class lookup of `x` in `p->~x()` and `decltype(...)::~x()`

- Ignored non-type `x` in `struct N::X x;`
- Limited `this/static` prohibition with “current class”
- Deprecated `A::template B` (as a template template argument or CTAD placeholder)
- Allowed forward declarations of partial specializations
- Removed mention of resolved issue
- Added explanatory examples to aid review

Introduction

The current descriptions of scope and name lookup are confusing, incomplete, and at times incorrect. [basic.scope] defines the word “scope” in two different ways, and then implicitly and indirectly specifies many aspects of name lookup with imperfect correspondence to the descriptions in [basic.lookup]. Lookup is often described for units of program text that may include multiple names and `<` tokens that must be disambiguated, such as a *template-id* ([temp.dep]/2), *id-expression* ([over.call.func]/2), or *conversion-type-id* ([class.qual]/1.2). Namespaces, and the global namespace in particular, are inconsistently described as being one or many declarative regions ([basic.scope.namespace]/1, /4). There are dozens of Core issues (discussed individually [below](#)) on the subject; the general vagueness of the wording on the subject interferes with resolving them individually.

Modules significantly complicate the situation, principally by invalidating the tacit assumption that “every” lookup is restricted to preceding declarations in the same translation unit. There were already exceptions to that rule (*e.g.*, complete class contexts and [using-declaration conflicts](#)), which can then easily be (mis)read as applying to all contents of all translation units. Other rules, like those for lookup through a *using-directive*, that do not make any mention of location may now plausibly be read as being available through an `import`.

This paper rewrites large sections of [basic.scope] and [basic.lookup] and parts of [temp.res] (and makes the requisite changes elsewhere to adapt to new definitions) to address these issues holistically. 51 Core issues are resolved, as well as 17 points discussed on the reflector but not (yet) on the issues list. Many changes are (collectively) editorial, but their very number demands CWG review. Other changes generally formalize current implementation practice or, in the absence of implementation consensus, establish language consistency.

Approach

To avoid redundant specification of scope and of name lookup, all description of where a name might be used ([basic.scope.block], [basic.scope.enum], *etc.*) is phrased merely in terms of the total extent of each scope where a name might be introduced. The scopes are always properly nested, although there are situations where unqualified lookup considers a scope that does not contain the point of the lookup. Explicit wording is introduced to require that lookup occur from a program point (replacing lookup “in the context” of something) and consider only previous declarations (except in complete-class contexts) and those made available by `import`. Most declarations are said to “bind” the names they introduce to themselves, replacing the notion of “not found by lookup”. To avoid confusion, the word “bind” is not used for describing locked-in lookup results as for default arguments (which can instead be described as being looked up from their declarations). The use of the word “visible” to indicate a declaration’s placement is avoided in normative text.

Several kinds of scope search are defined, and several kinds of qualified lookup are combined, to minimize redundancy in the definition of the lookup algorithms. The word “lookup” or phrase “look up” is used only for resolution of a name that is complete (at least in the simple context; it might be reused as a subroutine elsewhere).

The descriptions of the algorithms generally favor local completeness at the expense of additional references to distant clauses (that would otherwise specify special cases for their respective language features). Several rule restatements are turned into notes.

Every lookup is explicitly for a name, not an *id-expression* or *type-id*. Template arguments in an *id-expression* are addressed by defining which names are subject to qualified lookup and by excluding a *template-id* from being a name. Because labels interact with so little of the language, their identifiers are excluded from names and name lookup entirely.

To avoid confusion when a name is declared multiple times in a scope, careful distinction is made between a name and a declaration of that name. In particular, the “point of declaration” of a name (usually; [basic.scope.pdecl]/3 describes one for an anonymous class or enumeration) is replaced with the “locus” of a declaration.

The phrases “global name” and “global object” are removed to reduce the apparent special status of the global namespace (but “global scope” still exists because of its lack of syntax and as a shorthand).

Wording for the (possible) declaration of a class by an *elaborated-type-specifier* is removed from [basic.scope.pdecl]/7, [basic.lookup.elab]/2, [namespace.memdef]/3, and [class.friend]/11 and merged into [dcl.type.elab].

Richard's proposal for "[using-declarations & {,!}export](#)" is followed: a (non-dependent) *using-declaration* is replaced with its referent(s) during name lookup rather than introducing additional declarations. Because it is consistent with that replacement, and for consistency with existing implementations, the inclusion of hidden type names by a member *using-declaration* is extended to those not in a class.

Examples

The lookup for the following examples is narrated to help CWG members relate old and new terminology. Cross references prefixed with + are to N4849 as edited here.

using-directive vs. using-declaration

```
namespace F {float x;}
namespace A {
  namespace I {int x;}
  using namespace I;
  using F::x;
  void f() {
    x=0;    // #1: error
    A::x=1; // #2: OK: F::x
  }
}
```

1. At #1, `x` is looked up as an unqualified name (`+basic.lookup.unqual/4`) via an unqualified search of the function block scope of `A::f` (`/3`).
 2. When continuing on past the function parameter scope of `A::f` to the namespace scope of `A`, the unqualified search searches `A` and also every namespace contained by `A` and nominated by an active *using-directive* (`/2`).
 3. The single *using-directive* is active in the block scope (`/1`), which is contained by `A`, as is `I`.
 4. Therefore `A` and `I` are searched simultaneously.
 5. The *using-declaration* is replaced with `F::x` (`+basic.lookup/5`), and the lookup is ambiguous (`[basic.lookup/1]`).
-
1. At #2, `A` is looked up as an unqualified name. `x` is a qualified name (`+basic.lookup.qual/2`), so undergoes qualified name lookup (`/3`) which is just a search of `A`.
 2. The search finds `F::x`, so the *using-directive* is not considered by `+namespace.qual/1`.

CWG459

```
struct A {using T=int;}; // given
template<class T>
struct B : A {
  void f() {T i;}
};
```

1. `T` (in the *template-head*), `B`, and `f` are not looked up (`+dcl.meaning/3`, `+class.pre/1`).
2. `A` is looked up as an unqualified name (`+basic.lookup.unqual/4`), ignoring non-type names (`[class.derived/2]`).
3. `T` is looked up as an unqualified name when defining the class template `B` (`+temp.res/1`):
 1. Unqualified name lookup performs an unqualified search in the enclosing scope (`+basic.lookup.unqual/3`), which is the function block scope of `B::f` (`+basic.scope.block/2`).
 2. For each enclosing scope, an unqualified search is just a search except when considering a namespace (`+basic.lookup.unqual/2`); a search is just a single search except when considering a class (template) (`+class.member.lookup/1`). The function block scope is neither and has no binding for `T` (and the parameter scope (`+basic.scope.param/1`) has only `__func__`).
 3. The next enclosing scope is that of `B`; classes and class templates have scopes (`+basic.scope.class/1`). It is a class template, and a single search in it finds nothing (`[class.member.lookup/4]`), so its non-dependent base classes are considered (`+class.member.lookup/5`).
 4. A single search in the scope of `A` finds the *typedef-name*.
 5. The next enclosing scope is the template parameter scope to which the template parameter `T` belongs (`[basic.scope.temp/2]`), but it is never considered.

Calling a conversion function

```
auto x = s.operator R T::*();
```

1. `x` is not looked up (`+dcl.meaning/3`).

2. `s` is subject to normal unqualified lookup (`[basic.lookup.unqual]/4`).
3. The names used in the *conversion-type-id* are looked up “in the same fashion as the *conversion-function-id*” (`/5`).
4. While that lookup can’t happen yet, the *conversion-function-id* is a member-qualified name (`[basic.lookup.qual]/2`) and thus undergoes qualified lookup in the class of `s` (call it `S`; `/3`).
5. If `S::R` finds nothing, `R` is looked up as an unqualified name (`[basic.lookup.unqual]/5` again); `τ` is considered the same way (separately), except that non-type names are ignored because of the following `::` (`[basic.lookup.qual]/1`).
6. Once the type named by the *conversion-type-id* is known (`[class.conv.fct]/1`), the *conversion-function-id* is looked up as described above.

Calling destructors

```
struct A {using X=A;};
struct B : A {struct X {};};
void f(B *b) {
    b->A::~X();           // error: A::X not considered
}
```

1. At #1, `A` undergoes qualified lookup in `B` (`[basic.lookup.qual]/3`).
2. Because its *nested-name-specifier* is a (member-)qualified name (`/2`), `x` is looked up as `A` was (`/5.1.2`), which finds `B::x`.
3. Since that is not the lookup context `A`, `x` is then tried as an unqualified name (`/5.2`), but that finds nothing.

Issues

Definitions

[CWG554](#) is resolved by using the word “scope” *instead of* “declarative region”, consistent with its very common use in phrases like “namespace scope”. The other uses of “scope” are implied by the explicit unqualified lookup algorithm. [CWG2165](#) is resolved by removing the conflicting definition of it in `[basic.scope]`.

[CWG405](#) is resolved by stating that argument-dependent lookup (sometimes) occurs after an ordinary unqualified lookup (making statements like “finding a variable prevents argument-dependent lookup” formally correct).

[CWG36](#) (which has the dubious distinction of having the earliest recorded date) is resolved by limiting the restriction in question to class scopes, where ambiguous accessibility would be a problem, since a *using-declaration* does not introduce declarations.

[CWG418](#) is resolved by trivial rephrasing along with that necessary for the new *using-declaration* interpretation.

[CWG1291](#) and [CWG2396](#) are resolved by explicitly specifying how to look up names in a *conversion-type-id*.

[CWG600](#) is resolved by explaining that accessibility affects naming a member in the sense of the ODR.

My proposed regularization of *try-block* containment in “[Cases missed by P1825R0 \(Merged wording for P0527R1 \(Implicitly move from rvalue references in return statements\) and P1155R3 \(More implicit moves\)\)](#)” is included in rewording the relevant bullets.

Lookup completeness

[CWG191](#) and [CWG1200](#) are resolved by defining unqualified lookup in terms of every enclosing scope. [CWG536](#) (long partially resolved) is resolved by specifying that unqualified lookup occurs for any kind of *unqualified-id* as an *id-expression*. [CWG399](#) is resolved by explicitly appealing to the lookup for the last component of any suitable *nested-name-specifier*. [CWG562](#) is resolved by defining lookup as occurring from a program point.

[CWG1906](#) (currently closed as NAD, perhaps because its proposed resolution suffered in attempting to fit the structure of `[basic.lookup.unqual]` rejected by [CWG191](#)) and [CWG2370](#) are resolved by Rachel’s regularization suggested in “[Reconsidering CWG1906 and its friend CWG2370](#)”, which may be in conflict with the [CWG opinion from San Diego](#).

My proposed simplification for “[Missing associated namespace for member template arguments?](#)” is incorporated on the assumption that the failure to examine the namespace of an associated class is an oversight. (GCC and ICC, but not Clang or MSVC, accept the example in that message.)

[CWG1822](#) is resolved by specifying that the body of a lambda remains in the surrounding (function parameter) scope.

[CWG1894](#) and its duplicate [CWG2199](#) are resolved per Richard’s proposal for “[dr407 still leaves open questions about typedef / tag hiding](#)”, using generic conflicting-declaration rules even for typedef, and discarding a redundant *typedef-name* when looking up an *elaborated-type-specifier*.

[namespace.udir]/5 is simply removed; careful backward-lookup rules already allow such usage, and no implementation actually delays any declarations until the end of a *namespace-definition*.

Jens’ interpretation of “[On beyond CWG372](#)” is included as a note (cf. Richard’s [proposed restriction](#)). [CWG607](#) is resolved by looking up unqualified names in a *mem-initializer-id* from outside the parameter scope. [CWG1837](#) is resolved by restricting this to referring to the innermost enclosing class.

[CWG2007](#) is resolved by skipping unqualified lookup for operators that must be member functions. [CWG255](#) (partially resolved by [N3778](#) is resolved by generally specifying the restriction to usual deallocation functions.

Richard’s correction to template parameter scope for constrained template template parameters for “[CWG186 is NAD \(or ‘Resolved’\)](#)” is incorporated. His correction to function parameter scope for non-defining declarations in “[Using a function parameter in a requires clause](#)” is applied directly.

The missing point of declaration for a concept identified by Andrew in “[recursive concepts](#)” is supplied (without any prohibition on self-referential concepts).

Complete-class context

The intent for [CWG2335](#) (contra those of the older [CWG1890](#), [CWG1626](#), [CWG1255](#), and [CWG287](#)) is supported by retaining the unrestricted forward lookup in complete-class contexts (despite current implementation behavior for non-templates). [CWG361](#) is partially resolved by that confirmation; the rest must be addressed along with [CWG2335](#). [CWG2331](#) is resolved by defining lookup from complete-class contexts and out-of-line member definitions. The rest of [CWG2009](#) is resolved by making it ill-formed NDR for forward lookup outside a complete-class context to change the results (before overload resolution, to avoid differences in instantiation). [CWG325](#) is resolved by preventing forward lookup while first parsing a class from producing a *template-id* (and relying on the [CWG2009](#) resolution to ban a change of interpretation) while generalizing the *template* prefix to support lookup in an incomplete class. [CWG1635](#) is partially resolved by also making *template* default arguments a complete-class context.

Declaration matching

My clarification in “[Where can namespace-scope functions and variables be redeclared?](#)” is applied, resolving [CWG1821](#).

[CWG1820](#) is resolved by requiring that a qualified *declarator-id* declare an entity. [CWG386](#), [CWG1839](#), [CWG1900](#), and Richard’s observation in “[are non-type names ignored in a class-head-name or enum-head-name?](#)” are resolved by describing the limited lookup that occurs for a *declarator-id*, including the changes in Richard’s [proposed resolution for CWG1839](#) and rejecting the example from [CWG1477](#). Consistent linkage is preserved by strengthening [dcl.stc]/6. [CWG138](#) is resolved according to [N1229](#).

[CWG110](#) is resolved by reducing the restriction in [temp.pre] to a note (matching the behavior of GCC, Clang, and ICC).

Modules

Richard’s proposed resolution for “[missing restriction on private-module-fragment](#)” is incorporated.

Nathan’s question about `export using` in “[using-declarations & {,}export](#)” is avoided by not introducing synonyms that might or might not be exported. Richard’s [issue submission for using-declaration conflicts](#) is resolved by combining the generic notions of “conflicts with” and “precedes”.

Nothing is done about P1498R1, since its solution does not involve name lookup. For example, the intent is to disallow lookups that use a [using-declaration that names a declaration of a name with internal linkage](#) rather than the *using-declaration* itself.

Parsing

[CWG1771](#) is resolved by describing the lookup of only an *identifier*, or (per my proposal in “[Down with template!](#)”) the name in a recognized *template-id* (whether or not dependent), preceding a `::`. [CWG1828](#) is resolved by disfavoring the interpretation of `::` as an entire *nested-name-specifier*.

My proposal for [“Down with template!”](#) is incorporated by generalizing “type-id-only context” to “type-only context” and applying it to `template` as well as `typename`. [CWG1835](#) is resolved by removing the special provision for `->/ . identifier <` in `[basic.lookup.classref]/1`, requiring that `template` be used to refer even to a non-member template in that position. [CWG1908](#) is resolved per its suggestion of performing qualified lookup first and stopping if it produces a *template-id*, always taking `<` to indicate one (per Richard’s proposal for [“On beyond CWG1908”](#)). My further [proposal](#) for parsing a *qualified-id* in a dependent class member access is applied. My proposal for [“grammar ambiguities with non-simple template-ids”](#) is applied, allowing any kind of name to be considered to refer to a template, except for a *conversion-function-id* for which the rules would be more complicated despite never applying. [CWG1841](#) is resolved by considering the injected-class-name of a class template to refer to a template.

[CWG1616](#) is resolved by explicitly characterizing the forbidden case in terms of lookup results.

Templates

[CWG459](#) is not affected: base classes are still searched in unqualified lookup before template parameters are considered.

[CWG1936](#) is resolved by expanding the definition of “dependent name” to include qualified names (*i.e.*, names previously said to be “a member of an unknown specialization”). [CWG1500](#) is resolved by further expanding it to include any *conversion-function-id* with a dependent type (and clarifying that dependent lookup takes place after substitution). My proposed regularization of *conversion-type-id* interpretation (which includes resolving [CWG2413](#)) in [“Down with template!”](#) and proposed rule for considering conversion function templates in [“When are conversion function templates hidden?”](#) are incorporated. The assumption that a qualified dependent name names a type is disabled inside a `decltype` or array bound. [CWG1028](#) is resolved by moving the “lookup from first declaration” rule from `[temp.over.link]/5` to `[temp.res]/1`. [CWG1907](#) is resolved by forbidding dependence on default (template) arguments not found by *lookup* for a dependent name.

[CWG1829](#) is resolved by checking for direct members of a class that is the current instantiation.

[CWG1089](#) (as [reinterpreted](#) after [CWG1111](#)) is resolved by performing unqualified lookup for the first component of a *nested-name-specifier* in a class member access when a dependent type prevents finding anything. [CWG1478](#) is resolved per [my interpretation](#) as independently suggested by Richard in [“Implementation status of core issue 96”](#).

[CWG2070](#) is resolved by specifying that a dependent *using-declarator* is considered to name a constructor if and only if it ends with `B: :B`. [CWG852](#) is resolved by explicitly mentioning *using-declarations* in class templates (and making it diagnosable in most cases to refer to a class that is definitely not a base).

[CWG1545](#) is resolved by interpreting entire friend declarations upon instantiation of an enclosing template.

[CWG2213](#) is resolved by allowing an *elaborated-type-specifier* to contain a *simple-template-id* without `friend`.

Richard’s simplification in [“Deduction failures and template-ids”](#) is applied as part of regularizing the interpretation of function names.

Wording

Relative to N4849.

[intro.defs]

[defs.signature]

Change the definition (leaving the note):

- **signature**
 ⟨function⟩ name, parameter-type-list ([`dcl.fct`]), and enclosing namespace ~~(if any)~~

[defs.signature.templ]

Change:

- signature

(function template) name, parameter-type-list ([dcl.fct]), enclosing namespace (if any), return type, *template-head*, and trailing *requires-clause* ([dcl.decl]) (if any)

[lex.ext]

Change paragraph 2:

- A *user-defined-literal* is treated as a call to a literal operator or literal operator template ([over.literal]). To determine the form of this call for a given *user-defined-literal* *L* with *ud-suffix* *X*, first let *S* be the set of declarations found by unqualified lookup for the *literal-operator-id* whose literal suffix identifier is *X* is looked up in the context of *L* using the rules for unqualified name lookup ([basic.lookup.unqual]). Let *S* be the set of declarations found by this lookup. *S* shall not be empty.

[basic]

[basic.pre]

Change paragraph 4:

- A *name* is a use of an *identifier* ([lex.name]), *operator-function-id* ([over.oper]), *literal-operator-id* ([over.literal]), *or conversion-function-id* ([class.conv.fct]), *or template-id* ([temp.names]) that denotes to indicate an entity *or label* ([stmt.goto], [stmt.label]).

Change paragraph 5:

- Every name that denotes an entity is introduced by a *declaration*. Every name that denotes a label is introduced either by a *goto statement* ([stmt.goto]) or a *labeled statement* ([stmt.label]), which is a

1. declaration or member-declaration ([dcl.pre], [class.mem]),
2. init-declarator ([dcl.decl]),
3. identifier in a structured binding declaration ([dcl.struct.bind]) or namespace-alias-definition ([namespace.alias]),
4. condition with a declarator ([stmt.stmt]),
5. member-declarator ([class.mem]),
6. using-declarator ([namespace.udecl]),
7. parameter-declaration ([dcl.fct]),
8. type-parameter ([temp.param]),
9. elaborated-type-specifier that introduces a name ([dcl.type.elab]),
10. class-specifier ([class.pre]),
11. enum-specifier or enumerator-definition ([dcl.enum]),
12. exception-declaration ([except.pre]),
13. implicit declaration of an injected-class-name ([class.pre]).

[Note: The interpretation of a *for-range-declaration* produces one or more of the above ([stmt.ranged]). — end note]
An entity *E* is denoted by the name (if any) that is introduced by a declaration of *E* or by a *typedef-name* introduced by a declaration specifying *E*.

Remove bullet (9.4).

[basic.def.odr]

Change paragraph 4:

- A variable is named by an expression if the expression is an *id-expression* that denotes it. A variable *x* whose named appears as by a potentially-evaluated expression *e* [...]

[basic.scope]

[basic.scope.declarative]

Remove subclause, referring the old stable name to [basic.scope.scope] (added below). Remove the cross reference in [dcl.struct.bind]/1. Update the 2 surviving cross references (in [basic.def.odr]/9) to refer to [basic.scope.scope]. Replace the 9 surviving mentions of “declarative region” (in [basic.def.odr]/9, [expr.prim.id.unqual]/2, [expr.prim.lambda.capture]/7, and [namespace.def]/2) with “scope”.

General [basic.scope.scope]

Add subclause to front of [basic.scope]:

- The declarations in a program appear in a number of *scopes* that are in general discontinuous. The *global scope* contains the entire program; every other scope *S* is first introduced by a declaration, *parameter-declaration-clause*, statement, or *handler* (as described in the following subclauses of [basic.scope]) appearing in another scope which is thereby the *parent scope* of *S*. An *enclosing* scope at a program point is any scope that contains it; the smallest such scope is said to be the *immediate scope* at that point. A scope *intervenes* between a program point *P* and a scope *S* (that does not contain *P*) if it is or contains *S* but does not contain *P*.
- Every entity *belongs* to a single scope. An entity belongs to the *target* scope of the declaration that introduces it. Unless otherwise specified:
 1. A declaration *inhabits* the immediate scope at its locus ([basic.scope.pdecl]).
 2. A declaration’s target scope is the scope it inhabits, and
 3. Any names introduced by a declaration are *bound* to it in its target scope.

A scope that is or contains the scope to which an entity belongs is an enclosing scope for that entity. [Note: Special cases include that:

1. The declaration in a *template-declaration* inhabits the same scope as the *template-declaration*.
2. Friend declarations and declarations of qualified names and template specializations do not bind names ([dcl.meaning]); those with qualified names target a specified scope, and other friend declarations and certain *elaborated-type-specifiers* ([dcl.type.elab]) target a larger enclosing scope.
3. Block-scope extern declarations target a larger enclosing scope but bind a name in their immediate scope.
4. The names of unscoped enumerators are bound in the two smallest enclosing scopes ([dcl.enum]).
5. A class’s name is also bound in its own scope ([class.pre]).
6. The names of the members of an anonymous union are bound in the union’s parent scope ([class.union.anon]).

— end note]

- Two declarations *potentially conflict* if the same name denotes distinct entities via each and
 1. either declaration declares a *typedef-name*, or
 2. both entities are types, or
 3. neither entity is a type and either is not a function or function template, or
 4. neither entity is a variable, non-static data member, enumerator, function, or function template, or
 5. both entities are functions, or both are function templates, and they cannot be overloaded ([over.load]).

The program is ill-formed if a declaration potentially conflicts with another that

1. is reachable ([module.reach]) from it,
2. has the same target scope, and
3. appears in the same translation unit

unless either declares a name with internal linkage. [Note: Declarations might conflict even if neither binds a name. [Example:

```
struct A {
    friend short f(); // #1
};
struct B {
    friend float f(); // error: conflicts with #1
};
```

— end example] — end note] In general, a name can otherwise be bound to multiple declarations in a scope.

- When instantiating a templated entity ([temp.pre]), any scope *S* introduced by any part of the template definition is considered to be introduced by the instantiated entity and to contain the instantiations of any declarations that inhabit

[basic.scope.pdecl]

Change each paragraph 1–12:

1. The **point of declaration** for **locus of a named declaration** ([basic.pre]) that is a **declarator** is immediately after **its** complete declarator ([dcl.decl]) and before its initializer (if any), except as noted below. [Example:
[...] — end example]
2. [Note: A name from an outer scope remains visible up to the **point** **locus of the** declaration **of the name** that hides it. [Example:
[...] — end example] — end note]
3. The **point** **locus of** declaration for a class or class template first declared by a **class-specifier** is immediately after the **identifier** or **simple-template-id** (if any) in its **class-head** ([class.pre]). The **point** **locus of** declaration for an enumeration **an enum-specifier or opaque-enum-declaration** is immediately after the **identifier** (if any) in either its **enum-specifier** ([dcl.enum]) or its first **opaque-enum-declaration** ([dcl.enum]), whichever comes first. The **point** **locus of** declaration of an alias or alias template **an alias-declaration** is immediately follows the **defining-type-id** to which the alias refers **after it**.
4. The **point of declaration** **locus of** a **using-declarator** that does not name a constructor is immediately after the **using-declarator** ([namespace.udecl]).
5. The **point of declaration** for an enumerator **locus of an enumerator-definition** is immediately after its **enumerator-definition**. [Example:
[...] — end example]
6. [Note: After the **point of** declaration of a class member, the member name can be **looked up** **found** in the scope of its class. [Note: This is true even if the class is an incomplete class. [...] — end note]
7. [...] **The locus of an elaborated-type-specifier that is a declaration** ([dcl.type.elab]) **is immediately after it**.
8. The **point of declaration** for **locus of** an injected-class-name **declaration** ([class.pre]) is immediately following the opening brace of the class definition.
9. The **point** **locus of** the **implicit** declaration **for of** a function-local predefined variable ([dcl.fct.def.general]) is immediately before the **function-body** of **its** function's definition.
10. The **point** **locus of** the declaration of a structured binding ([dcl.struct.bind]) is immediately after the **identifier-list** of the structured binding declaration.
11. The **point of declaration** for the variable or the structured bindings declared in the **locus of a** for-range-declaration of a range-based for statement ([stmt.ranged]) is immediately after the **for-range-initializer**.
12. The **point of declaration** for a template parameter **locus of a template-parameter** is immediately after its **complete template-parameter**. [Example:
[...]
— end example]

Insert before paragraph 13:

- The locus of a **concept-definition** is immediately after its **concept-name** ([temp.concept]). [Note: The **constraint-expression** cannot use the **concept-name**. — end note]

[Drafting note: There is no normative wording yet corresponding to this note. — end drafting note]

Change paragraph 13:

- [Note: Friend declarations **refer to** **may introduce** functions or classes that **are members of** **belong to** the nearest enclosing namespace **or block scope**, but they do not **introduce new** **bind** names **into that namespace anywhere** ([namespace.memdef.class.friend]). Function declarations at block scope and variable declarations with the extern specifier at block scope **refer to** declarations that are members of **an** **declare entities that belong to the nearest** enclosing namespace, but they do not **introduce new** **bind** names **into that scope in it**. — end note]

[basic.scope.block]

Replace contents, retaining the example:

- Each

1. selection or iteration statement ([stmt.select], [stmt.iter]),
2. substatement of such a statement,
3. *handler* ([except.pre]), or
4. compound statement ([stmt.block]) that is not the *compound-statement* of a *handler*,

introduces a *block scope* that includes that statement or *handler*. [Note: A substatement that is also a block has only one scope. — end note] A variable that belongs to a block scope is a *local variable*.

Move the example from [stmt.for]/3 to here.

- If a declaration whose target scope is the block scope *S* of a

1. *compound-statement* of a *lambda-expression*, *function-body*, or *function-try-block*,
2. substatement of a selection or iteration statement, or
3. *handler* of a *function-try-block*

potentially conflicts with a declaration whose target scope is the parent scope of *S*, the program is ill-formed.

[Example:

[...]

— end example]

[basic.scope.param]

Replace contents:

- A *parameter-declaration-clause* *P* introduces a *function parameter scope* that includes *P*. [Note: A function parameter cannot be used for its value within the *parameter-declaration-clause* ([dcl.fct.default]). — end note] If *P* is associated with a *declarator* and is preceded by a (possibly-parenthesized) *noptr-declarator* of the form *declarator-id attribute-specifier-seq_{opt}*, its scope extends to the end of the nearest enclosing *init-declarator*, *member-declarator*, *declarator* of a *parameter-declaration* or a *nodeclspec-function-declaration*, or *function-definition*. [Note: In this case, *P* declares the parameters of a function (or a function or template parameter declared with function type). A member function's parameter scope is nested within its class's scope. — end note] If *P* is associated with a *lambda-declarator*, its scope extends to the end of the *compound-statement* in the *lambda-expression*. If *P* is associated with a *requirement-parameter-list*, its scope extends to the end of the *requirement-body* of the *requires-expression*. If *P* is associated with a *deduction-guide*, its scope extends to the end of the *deduction-guide*.

[basic.funscope]

Remove subclause, referring the stable name (which has no cross references) to [stmt.label].

[basic.scope.namespace]

Replace contents:

- Each namespace *N* introduces a *namespace scope* that includes the *namespace-body* for every *namespace-definition* for *N*. The scope also includes the portion of any redeclaration or specialization of an entity enclosed by it after the *declarator-id*, *class-head-name*, or *enum-head-name* that names the entity. The global scope is the namespace scope of the global namespace ([basic.namespace]).

[basic.scope.class]

Replace contents:

- Each class or class template *C* introduces a *class scope* that includes the *class-specifier* for *C*. The scope also includes the portion of any redeclaration or specialization of an entity *E* enclosed by it after the *declarator-id*, *class-head-name*,

or *enum-head-name* that names *E*. [Note: Lookup from a program point before the *class-specifier* of a class will find no bindings in the class scope. [Example:

```
template<class D>
struct B {
    D::type x; // error: C::type not found
};

struct A {using type=int;};
struct C : A,B<C> {};
```

— end example] — end note]

[basic.scope.enum]

Replace contents:

- Each enumeration *E* introduces an *enumeration scope* that includes the *enum-specifier* for *E*.

[basic.scope.temp]

Replace contents:

- Each template *template-parameter* introduces a *template parameter scope* that includes the *template-head* of the *template-parameter*.
- Each *template-declaration* *D* introduces a template parameter scope that includes it. Any declaration outside the *template-parameter-list* that would inhabit that scope instead inhabits the same scope as *D*. [Note: Therefore, only template parameters belong to a template parameter scope. — end note]

[basic.scope.hiding]

Remove subclause, referring the stable name to [basic.lookup].

[basic.lookup]

Change paragraph 1:

- [...] Name lookup associates the use of a name with a set of declarations ([basic.def]) of that name. Unless otherwise specified, the program is ill-formed if no declarations are found. If the declarations found by name lookup all denote functions or function templates, the declarations are said to form an *overload set*. Otherwise, if the declarations found by name lookup shall either do not all denote the same entity or form an overload set, they are ambiguous and the program is ill-formed. Overload resolution ([over.match], [over.over]) takes place after name lookup has succeeded. The access rules ([class.access]) are considered only once name lookup and function overload resolution (if applicable) have succeeded. Only after name lookup, function overload resolution (if applicable) and access checking have succeeded are the semantic properties introduced by the name's declaration and its reachable ([module.reach]) redeclarations used further in expression processing ([expr]).

Insert before paragraph 2:

- A program point *P* follows any declaration in the same translation unit whose locus ([basic.scope.pdecl]) is before *P*. [Note: The declaration might appear in a scope that does not contain *P*. — end note]
- A declaration *X* precedes a program point *P* in a translation unit *L* if *P* follows *X*, or else *X* appears in a translation unit *D* and
 1. *P* follows a *module-import-declaration* or *module-declaration* that imports *D* (directly or indirectly), and
 2. *X* appears after the *module-declaration* in *D* (if any) and before the *private-module-fragment* in *D* (if any), and
 3. either *X* is exported or *D* and *L* are part of the same module, and
 4. *X* does not declare a name with internal linkage. [Note: Names declared by a *using-declaration* have no linkage. — end note]

Move the note (and example) from [basic.scope.namespace]/2 to the end of this paragraph.

Remove paragraphs 2 through 4.

Append paragraphs:

- A *single search* in a scope *S* for a name *N* from a program point *P* finds the declarations in *S* that precede *P* and to which any name that is the same as *N* ([basic.pre]) is bound. If any such declaration is a *using-declarator* that is not dependent ([temp.dep.type]), it is replaced by the declarations named by the *using-declarator* ([namespace.udecl]). If the containing *using-declaration* is a *member-declaration*, the replacement declarations are considered to have the access of the *using-declaration*.
- In certain contexts, only certain kinds of declarations are included. After any such restriction, any declarations of classes or enumerations are discarded if any other declarations are found. [Note: A type (but not a *typedef-name* or template) is therefore hidden by any other entity in its scope. — end note] However, if a lookup is *type-only*, only declarations of types and templates whose specializations are types are considered; furthermore, if a *typedef declarator* and a declaration of the type to which it refers are found, the *typedef declarator* is discarded instead of the type declaration.
- If *N* is a *conversion-function-id*, conversion function templates that belong to *S* are considered. For each such template, if template argument deduction succeeds ([temp.deduct.conv]), an invented declaration of the resulting specialization is included in the result.

[class.member.lookup]

Move to front of [basic.lookup], editing as follows:

Remove paragraph 1.

Change paragraph 2:

- A search in a scope *C* for a name *N* from a program point *P* is a single search in *C* for *N* from *P* unless *C* is a class or class template, in which case the following steps define the result of a member name *f* in a class scope *C* the search. [Note: The result differs only if the single search would find nothing. — end note]

Replace *f* with *N* and *C* with *C* in the following paragraphs.

Change paragraph 3:

- The *lookup set* for *f* in *C*, called *S*(*f*, *C*), consists of two component sets: the *declaration set*, a set of members named *f*; and the *subject set*, a set of subobjects where declarations of these members (possibly including *using-declarations*) were found. (possibly via *using-declarations*). In the declaration set, *using-declarations* are replaced by the set of designated members that are not hidden or overridden by members of the derived class ([namespace.udecl]), and type declarations (including injected-class-names) are replaced by the types they designate. *S*(*f*, *C*) is calculated as follows:

Change paragraph 4:

- ~~If *C* contains a declaration of the name *f*, the declaration set contains every declaration of *f* declared in *C* that satisfies the requirements of the language construct in which the lookup occurs.~~ The declaration set is the result of a single search in *C* for *N* from immediately after the class-specifier of *C* if *P* is in a complete-class context of *C* or from *P* otherwise. [Note: [...] — end note][...]

[Drafting note: The plan for CWG2335 is to describe forbidden dependency cycles among the complete-class contexts of a class. — end drafting note]

Change paragraph 5:

- [...] ~~If *C* has base classes, e~~ Calculate the lookup set for *f* in each direct non-dependent ([temp.dep.type]) base class subobject *B_i*, and merge each such lookup set *S*(*f*, *B_i*) in turn into *S*(*f*, *C*). [Note: If *C* is incomplete, only base classes whose *base-specifier* appears before *P* are considered. If *C* is an instantiated class, its base classes are not dependent. — end note]

Change paragraph 7:

- The result of name lookup for t in C is the declaration set of $S(\{N, C\})$. If it is an invalid set, the program is ill-formed. If the use of N is followed by a $<$ that is not interpreted as part of a template argument list ([temp.names]) and the result contains any declaration of a template, the program is ill-formed, no diagnostic required. If it differs from the result of a search in C for N from immediately after the *class-specifier* of C , the program is ill-formed, no diagnostic required.

Remove paragraph 8.

[basic.lookup.unqual]

Replace contents (including the existing paragraph 16 at the end):

- A *using-directive* is active in a scope S at a program point P if P appears later in the same translation unit and it inhabits S or to the scope of a namespace nominated by a *using-directive* that is active in S at P .
- An *unqualified search* in a scope S from a program point P includes the results of searches from P in
 1. S , and
 2. for any scope U that contains P and is or is contained by S , each namespace contained by S that is nominated by a *using-directive* that is active in U at P .

If no declarations are found, the unqualified search also includes an unqualified search in the parent scope of S , if any, from P . [Note: When a class scope is searched, the scopes of its base classes are also searched ([class.member.lookup]). If it inherits from a single base, it is as if the scope of the base immediately contains the scope of the derived class. — end note]

- *Unqualified name lookup* from a program point performs an unqualified search in its immediate scope.
- An *unqualified name* is a name that does not follow a *nested-name-specifier* or the `.` or `->` in a class member access expression ([expr.ref]), possibly after a `template` keyword or `~`. Unless otherwise specified, such a name undergoes unqualified name lookup from the point where it appears.
- An unqualified name that appears in a *conversion-type-id* is looked up in the same fashion as the *conversion-function-id* in which it appears. If that lookup finds nothing, it undergoes unqualified name lookup; in each case, only names that denote types or templates whose specializations are types are considered.
- In a friend declaration, lookup for an unqualified name that appears in a *declarator* after a *declarator-id* with a *nested-name-specifier* that names a class, class template, or namespace S performs an unqualified search in S . If that lookup finds nothing, it undergoes unqualified name lookup.
- [Note: The rules for name lookup in template definitions are described in [temp.res]. — end note]

[basic.lookup.argdep]

Change paragraph 1:

- When the *postfix-expression* in a function call ([expr.call]) is an *unqualified-id*, and other namespaces not considered during the usual unqualified lookup ([basic.lookup.unqual]) may be searched, and in those namespaces, namespace-scope friend function or function template declarations ([class.friend]) not otherwise visible may be found. These modifications to the search for the name in the *unqualified-id* does not find any
 1. declaration of a class member, or
 2. function declaration inhabiting a block scope, or
 3. declaration not of a function or function template

then lookup for the name also includes the result of argument-dependent lookup in a set of associated namespaces that depends on the types of the arguments (and for template template arguments, the namespace of the template argument), as specified below. [Example:

[...]

— end example]

Move the note from [basic.lookup.unqual]/3 here, inserting it before paragraph 2.

Change paragraph 2:

- For each argument type τ in the function call, there is a set of zero or more associated namespaces and a set of zero or more associated entities (other than namespaces) to be considered. The sets of namespaces and entities are determined entirely by the types of the function arguments (and the namespace of any template template arguments). Typedef names and *using-declarations* used to specify the types do not contribute to this set. The sets of namespaces and entities are determined in the following way:

1. If τ is a fundamental type, its associated sets of namespaces and entities are both empty.
2. If τ is a class type (including unions), its associated entities are: the class itself; the class of which it is a member, if any; and its direct and indirect base classes. Its associated namespaces are the innermost enclosing namespaces of its associated entities. Furthermore, if τ is a class template specialization, its associated namespaces and entities also include: the namespaces and entities associated with the types of the template arguments provided for template parameters (excluding template template parameters); the templates used as template template arguments; the namespaces of which any template template arguments are members; and the classes of which any member templates used as template template arguments are members. [Note: Non-type template arguments do not contribute to the set of associated namespaces entities. — end note]
3. If τ is an enumeration type, its associated namespace is the innermost enclosing namespace of its declaration, and its associated entities are τ and, if it is a class member, the member's class.
4. If τ is a pointer to U or an array of U , its associated namespaces and entities are those associated with U .
5. If τ is a function type, its associated namespaces and entities are those associated with the function parameter types and those associated with the return type.
6. If τ is a pointer to a member function of a class X , its associated namespaces and entities are those associated with the function parameter types and return type, together with those associated with X .
7. If τ is a pointer to a data member of class X , its associated namespaces and entities are those associated with the member type together with those associated with X .

If an associated namespace is an inline namespace (`[namespace.def]`), its enclosing namespace is also included in the set. If an associated namespace directly contains inline namespaces, those inline namespaces are also included in the set. In addition, if the argument is the name or address of an overload set or the address of such a set, its associated entities and namespaces are the union of those associated with each of the members of the set, i.e., the entities and namespaces associated with its parameter types and return type. Additionally, if the aforementioned overload set is named with a *template-id*, its associated entities and namespaces also include those of its type template arguments and its template template arguments and those associated with its type template arguments.

Change paragraph 3 and move it to the end of the subclause:

- Let X be the lookup set produced by unqualified lookup (`[basic.lookup.unqual]`) and let Y be the lookup set produced by argument dependent lookup (defined as follows). If X contains

1. a declaration of a class member, or
2. a block-scope function declaration that is not a *using-declaration*, or
3. a declaration that is neither a function nor a function template

then Y is empty. Otherwise Y is the set of declarations found in the namespaces associated with the argument types as described below. The set of declarations found by the lookup of the name is the union of X and Y . [Note: The associated namespaces and entities associated with the argument types can include namespaces and entities already considered by the ordinary unqualified lookup. — end note] [Example:

[...]

— end example]

Change paragraph 4:

- When considering an associated namespace for a call are the innermost enclosing non-inline namespaces for its associated entities. Argument-dependent lookup finds the results of qualified name lookup in each associated namespace N , the lookup is the same as the lookup performed when N is used as a qualifier (`[namespace.qual]`) except that:

1. Any *using-directives* in N are ignored.
2. All names except those *Only declarations* of (possibly overloaded) functions and function templates are ignored considered.

3. Any namespace-scoped declaration of a friend functions or friend function templates ([class.friend]) declared in a classes with a reachable definitions in the set of associated entities are visible within their respective namespaces even if they are not visible during an ordinary lookup ([namespace.memdef]) is considered to precede the point of the lookup, and the name declared is considered to be bound to it in its target scope.
4. Any exported declaration D in N declared within the purview of a named module M ([module.interface]) is visible considered to precede the point of the lookup P if it does not appear in the translation unit containing P and there is an associated entity attached to M with the same innermost enclosing non-inline namespace scope as D .
5. If the lookup is for a dependent name ([temp.dep], [temp.dep.candidate]), any declaration D in N is visible if D would be visible to qualified name lookup ([namespace.qual]) at any the above lookup is also performed from each point in the instantiation context ([module.context]) of the lookup, unless D is declared additionally ignoring any declaration that appears in another translation unit, is attached to the global module, and is either discarded ([module.global.frag]) or has internal linkage.

[basic.lookup.qual]

Change paragraph 1:

- The name of a class or namespace member or enumerator can be referred to after the `::` scope resolution operator ([expr.prim.id.qual]) applied to a nested-name-specifier that denotes its class, namespace, or enumeration. If Lookup of an identifier, or the name in a template-id, followed by a `::` scope resolution operator in a nested-name-specifier is not preceded by a decltype-specifier, lookup of the name preceding that `::` considers only namespaces, types, and templates whose specializations are types. [Note: An identifier followed by `<` is not considered to be part of a template-id during a lookup to determine the interpretation of the `<` ([temp.names]). — end note] If the name found does not a name, template-id, or decltype-specifier is followed by a `::`, it shall designate a namespace or a class, enumeration, or dependent type, the program is ill-formed and the `::` is never interpreted as a complete nested-name-specifier. [Example:

[...]

— end example]

Insert before paragraph 2:

- A member-qualified name is an unqualified-id, or a nested-name-specifier of the form `type-name ::` or `namespace-name ::`, in the `id-expression` of a class member access expression ([expr.ref]). A qualified name is a member-qualified name, a qualified-id, a using-declarator, a typename-specifier, or a nested-name-specifier, elaborated-type-specifier, or class-or-decltype that contains a nested-name-specifier ([expr.prim.id.qual]). The lookup context of a member-qualified name is the type of its associated object expression (considered dependent if the object expression is type-dependent). The lookup context of any other qualified name is the type or namespace nominated by its nested-name-specifier. The terminal name of a qualified name is the identifier (including that of a simple-template-id) in it or the (initial) name in the unqualified-id that is or is in the qualified name. [Note: When parsing a class member access, the name following the `->` or `.` is a terminal name even though it is not yet known of which kind. — end note]
- Qualified name lookup in a class, namespace, or enumeration performs a search of the scope associated with it ([class.member.lookup]) except as specified below. Unless otherwise specified, a terminal name undergoes qualified name lookup in the lookup context of its qualified name from the point where it appears unless the lookup context either is dependent and is not the current instantiation ([temp.dep.type]) or is not a class or class template. If nothing is found by qualified lookup for the terminal name of a member-qualified name that is a nested-name-specifier, the terminal name undergoes unqualified lookup. [Example:

```
int f();
struct A {
    template<int> using B=A;
    int C;
    template<int> using D=void;
    typedef A E,T;
    void f();
};
template<int> using C=A;
template<int> using D=A;
template<int> using E=A;

template<class T>
void g(T *p) {
```

```

p->template B<0>::f(); // error: B not found and not dependent
p->T::template B<0>::f(); // OK: T is the template parameter; A::B found in f<A>
p->T::T::f(); // OK: second T is A::T in f<A>
p->C<0>::f(); // OK: (p->C < 0) > ::f()
p->template D<0>::f(); // OK: ::D found in definition context
}
template void g(A*);

void h(A a) {
    a.template C<0>::f(); // OK: non-type A::C ignored
    a.template E<0>::f(); // error: E is A::E which is not a template
}

```

— end example]

Remove paragraphs 3 through 5.

Change paragraph 6:

- In a qualified-id of the form:

nested-name-specifier_{opt} type-name :: type-name

the second type-name is looked up in the same scope as the first. If the qualified name is or contains an unqualified-id that begins with ~ identifier:

1. If the qualified name is not a member-qualified name:
 1. Its nested-name-specifier shall nominate a type.
 2. If the nested-name-specifier is a qualified name, the identifier is looked up in the same fashion as its terminal name.

[Note: Otherwise, the qualified name is looked up normally. — end note]
2. If lookup for the identifier finds nothing or finds it to be a type-name that does not refer to the lookup context (ignoring cv-qualification), it instead undergoes unqualified lookup.
3. Each lookup for the identifier considers only types (if the identifier is not followed by a <) and templates whose specializations are types.
4. The type-name that is or contains the identifier shall ultimately refer to the lookup context (ignoring cv-qualification).

[Example:

```

struct C {
    typedef int I;
    typedef float I2;
};
typedef int I1, I2;
extern int* p;
extern int* q;
void f() {
    p->C::I::~I2(); // after finding C::I2 is, unqualified looked up in the scope of C finds ::I2
    q->I1::~I2(); // I2 is undergoes unqualified looked up in the scope of the postfix expression
}

struct A {
    ~A();
};
typedef A AB;
int main() {
    AB* p;
    p->AB::~~AB(); // explicitly calls the destructor for A
}

```

— end example] [Note: [basic.lookup.classref] describes how name lookup proceeds after the ~- and -> operators.
— end note]

[class.qual]

Remove paragraph 1.

Change paragraph 2:

- In a lookup in which function names are not ignored^[...] and the for a qualified name with a nested-name-specifier that nominates a class C:

1. if the name specified after the nested-name-specifier, when looked up in C, is search finds the injected-class-name of C ([class.pre]), or
2. if the qualified name is a using-declarator of a using-declaration ([namespace.udecl]) that is a member-declaration, if the name specified after the nested-name-specifier is the same as the identifier or the simple-template-id's template-name in the last component of the nested-name-specifier dependent and names a constructor,

the name is instead considered to name the constructor of class C. [Note: For example, the constructor is not an acceptable lookup result in an elaborated-type-specifier so the constructor would not be used in place of the injected-class-name. — end note] Such a constructor name shall be used only in the declarator-id of a declaration that names a constructor or in a using-declaration. [Example:

[...]

— end example]

Remove paragraph 3.

[namespace.qual]

Remove paragraphs 1 and 2.

Replace paragraph 3, retaining its example:

- Qualified name lookup in a namespace *N* additionally searches every element of the inline namespace set of *N* ([namespace.def]), ignoring unnamed namespaces defined in translation units that do not contain the point of the lookup ([namespace.unnamed]). If nothing is found, it includes the results of qualified name lookup in each namespace nominated by a *using-directive* that is followed by the point of the lookup and inhabits *N* or an element of its inline namespace set. [Note: If a *using-directive* refers to a namespace that has already been considered, it does not affect the result. — end note] [Example:

[...]

— end example]

Change paragraph 6:

- During the lookup of a qualified namespace member name, if the lookup finds more than one declaration of the member, and if one declaration introduces a *class name* or *enumeration name* and the declarations are not discarded because of other declarations introduce either the same variable, the same enumerator, or a set of functions, the non-type name hides the class or enumeration name if and only if the declarations are from the same namespace; otherwise (the declarations are from different namespaces), the program is ill-formed found in other searches. — end note] [Example:

[...]

— end example]

Remove paragraph 7.

[basic.lookup.elab]

Remove paragraphs 1 and 2.

Insert before paragraph 3:

- If the *class-key* or *enum* keyword in an *elaborated-type-specifier* is followed by an *identifier* that is not followed by ::, lookup for the *identifier* is type-only. [Note: In general, the recognition of an *elaborated-type-specifier* depends on the following tokens. If the *identifier* is followed by ::, see [basic.lookup.qual]. — end note]

Change paragraph 3:

- If the *elaborated-type-specifier* has a *nested-name-specifier*, *qualified name* lookup is performed, as described in [basic.lookup.qual], but ignoring any non-type names that have been declared for the following *identifier* is type-only. If the name lookup does not find a previously declared *type-name*, the *elaborated-type-specifier* is ill-formed.

[basic.lookup.classref]

Remove subclause, referring the stable name to [basic.lookup.qual].

[basic.link]

Change paragraph 4:

- A *The* name *having of an entity that belongs to a* namespace scope ([basic.scope.namespace]) has internal linkage if it is the name of
- [...]

Change paragraph 7:

- The name of a function declared in block scope and the name of a variable declared by a block scope *extern* declaration have linkage. If such a declaration is attached to a named module, the program is ill formed. If there is a visible declaration of an entity with linkage, ignoring entities declared outside the innermost enclosing namespace scope, such that the block scope declaration would be a (possibly ill-formed) redeclaration if the two declarations appeared in the same declarative region, the block scope declaration declares that same entity and receives the linkage of the previous declaration. If there is more than one such matching entity, the program is ill-formed. Otherwise, if no matching entity is found, the block scope entity receives external linkage. If, within a translation unit, the same entity is declared with both internal and external linkage, the program is ill-formed. [Example:

```
static void f();
extern "C" void h();
static int i = 0;           // #1
void g() {
    extern void f();        // internal linkage
    extern void g();        // ::g, external linkage
    extern void h();        // C language linkage
    int i;                 // #2: i has no linkage
    {
        extern void f();    // internal linkage
        extern int i;       // #3: exinternal linkage, ill formed
    }
}
```

Without *Even though* the declaration at line #2 *hides the declaration at line #1*, the declaration at line #3 *would link with the still* *redeclarations* at line #1. Because the declaration with *and receives* internal linkage *is hidden, however, #3 is given external linkage, making the program ill-formed.* — end example]

Remove paragraph 8 (whose example is moved to [dcl.meaning]).

Change bullet (11.2):

- both names refer to *members of entities that belong to* the same namespace or *to members, not by inheritance, of the same* class; and

[basic.life]

Change the footnote in paragraph 6:

- For example, before the *construction of a global object that is initialized via a user-provided constructor* *dynamic initialization of an object with static storage duration* ([class.editor] *basic.start.dynamic*)).

[basic.type.qualifier]

Change paragraph 3:

- See [dcl.fct] and [*class.this* *over.match.funcs*] regarding function types that have *cv-qualifiers*.

[expr]

[expr.prim]

[expr.prim.this]

Change paragraph 1:

- The keyword `this` names a pointer to the object for which a non-static member function ([class.~~this~~ mfct.non-static]) is invoked or a non-static data member's initializer ([class.mem]) is evaluated.

Insert before paragraph 2:

- The *current class* at a program point is the class associated with the smallest class scope containing that point. [Note: A *lambda-expression* does not contain any class scope. — end note]

Change paragraph 2:

- If a declaration declares a member function or member function template of a class `X`, the expression `this` is a prvalue of type “pointer to *cv-qualifier-seq* `X`” wherever `X` is the current class between the optional *cv-qualifier-seq* and the end of the *function-definition*, *member-declarator*, or *declarator*. It shall not appear before the optional *cv-qualifier-seq* and it shall not appear within the declaration of a static member function of the current class (although its type and value category are defined within a static member function as they are within a non-static member function). [Note: This is because declaration matching does not occur until the complete declarator is known. — end note] [Note: In a *trailing-return-type*, the class being defined is not required to be complete for purposes of class member access ([expr.ref]). Class members declared later are not visible. [Example:

[...]

— end example] — end note]

Change paragraph 3:

- Otherwise, if a *member-declarator* declares a non-static data member ([class.mem]) of a class `X`, the expression `this` is a prvalue of type “pointer to `X`” wherever `X` is the current class within the optional default member initializer ([class.mem]). ~~It shall not appear elsewhere in the *member-declarator*.~~

[expr.prim.id]

[expr.prim.id.unqual]

Change paragraph 2:

- The result is the entity denoted by the ~~identifier~~ unqualified-id ([basic.lookup.unqual]). If the entity is a local entity [...]

[expr.prim.id.qual]

Change paragraph 1:

- The *nested-name-specifier* `::` nominates the global namespace. The type denoted by a *nested-name-specifier* with a *decltype-specifier* in a *nested-name-specifier* nominates the type denoted by the *decltype-specifier*, which shall be a class or enumeration type. Any other *nested-name-specifier* nominates the entity denoted by its *type-name*, *namespace-name*, or *terminal name*. [Note: In a *template-declaration*, a *type-name* that is a *simple-template-id* can denote a *template* ([temp.dep.type]). — end note]

Change paragraph 2:

- ~~A *nested-name-specifier* that denotes a class, optionally followed by the keyword `template` ([temp.names]), and then followed by the name of a member of either that class ([class.mem]) or one of its base classes ([class.derived]), is a *qualified-id*; [class.qual] describes name lookup for class members that appear in *qualified-ids*. The result is the member entity denoted by the *qualified-id* ([basic.lookup.qual]). The type of the ~~result expression~~~~ is the type of the

~~member result~~. The result is an lvalue if the member is a function, a variable, a structured binding ([dcl.struct.bind]), a static member function, or a data member and a prvalue otherwise. ~~[Note: A class member can be referred to using a qualified-id at any point in its potential scope ([basic.scope.class]). — end note]~~ Where *type-name ::= type-name* is used, the two *type-names* shall refer to the same type (ignoring cv-qualifications); this notation denotes the destructor of the type so named ([expr.prim.id.dtor]). The *unqualified-id* in a *qualified-id* shall not be of the form *~decltype-specifier* or (if the *nested-name-specifier* in the *qualified-id* is of the form *decltype-specifier::*) the form *~type-name*.

Remove paragraphs 3 through 5.

[expr.prim.lambda]

[expr.prim.lambda.closure]

Change paragraph 4:

- [...] ~~[Note: Names referenced in the *lambda-declarator* are looked up in the context in which the *lambda-expression* appears. — end note]~~ [Example:

[...]

— end example]

Change paragraph 12:

- The *lambda-expression*'s *compound-statement* yields the *function-body* ([dcl.fct.def]) of the function call operator, but for purposes of name lookup ([basic.lookup]), determining the type and value of *this* ([class.this]) and transforming *id-expressions* referring to non-static class members into class member access expressions using *(*this)* ([class.mfct.non-static]), the *compound-statement* is considered in the context of the *lambda-expression* it is not within the scope of the closure type. [Example:

[...]

— end example] Further, a variable `__func__` is implicitly defined at the beginning of the *compound-statement* of the *lambda-expression*, with semantics as described in [dcl.fct.def.general].

[expr.prim.lambda.capture]

Change paragraph 6:

- An *init-capture* behaves as if it ~~declares and~~ explicitly captures a variable of the form “`auto init-capture ;`” ~~whose declarative region is~~ declared in the *lambda-expression*'s *compound-statement*, except that:

[...]

Change paragraph 17:

- A *simple-capture* followed by an ellipsis is a pack expansion ([temp.variadic]). An *init-capture* preceded by an ellipsis is a pack expansion that ~~introduces~~ declares an *init-capture* pack ([temp.variadic]) ~~whose declarative region is~~ in the *lambda-expression*'s *compound-statement*. [Example:

[...]

— end example]

[expr.prim.req]

Change the grammar in paragraph 1:

- [...]

requirement-parameter-list:

(*parameter-declaration-clause* ~~opt~~)

[...]

Change paragraph 4:

- A *requires-expression* may introduce local parameters using a *parameter-declaration-clause* ([dcl.fct]). A local parameter of a *requires-expression* shall not have a default argument. Each name introduced by a local parameter is in scope from the point of its declaration until the closing brace of the *requirement-body*. These parameters have no linkage, storage, or lifetime; they are only used as notation for the purpose of defining requirements. [...]

Remove paragraph 5.

[expr.post]

[expr.call]

Change paragraph 7:

- [...] If the function is a non-static member function, the *this* parameter of the function ([class.expr.prim.this]) is initialized with a pointer to the object of the call, converted as if by an explicit type conversion ([expr.cast]). [...]

[expr.ref]

Change paragraph 1:

- A postfix expression followed by a dot `.` or an arrow `->`, optionally followed by the keyword `template` ([temp.names]), and then followed by an *id-expression*, is a postfix expression. The postfix expression before the dot or arrow is evaluated; [...] the result of that evaluation, together with the *id-expression*, determines the result of the entire postfix expression. [Note: If the keyword `template` is used, the following unqualified name is considered to refer to a template ([temp.names]). If a *simple-template-id* results and is followed by a `::`, the *id-expression* is a *qualified-id*. — end note]

Change paragraph 4:

- Otherwise, the object expression shall be of class type. The class type shall be complete unless the class member access appears in the definition of that class. [Note: If the class is incomplete, lookup in The program is ill-formed if the result differs from that when the class is complete. class type is required to refer to the same declaration ([basic.scope.class.member.lookup]). — end note] The *id-expression* shall name a member of the class or of one of its base classes. [Note: Because the name of a class is inserted in its class scope ([class]), the name of a class is also considered a nested member of that class. — end note] [Note: [basic.lookup.classref.qual] describes how names are looked up after the `.` and `->` operators. — end note]

[expr.unary]

[expr.await]

Change bullet (3.2):

- *a* is the *cast-expression* if Unless the *await-expression* was implicitly produced by a *yield-expression* ([expr.yield]), an initial suspend point, or a final suspend point ([dcl.fct.def.coroutine]); Otherwise, a search is performed for the unqualified *id-name* `await_transform` is looked up within the scope of *P* by class member access lookup ([basic.class.member.lookup.classref]), and *i*. If this lookup search is performed and finds at least one declaration, then *a* is *p.await_transform(cast-expression)*; otherwise, *a* is the *cast-expression*.

[expr.new]

Change paragraph 11:

- If the *new-expression* does not begin with a unary `::` operator, the allocation function's name is looked up in the global scope. Otherwise, if and the allocated type is a class type *T* or array thereof, a search is performed for the allocation function's name is looked up in the scope of *T*. Otherwise, or if this lookup fails to find the name, or if the

allocated type is not a class type **nothing is found**, the allocation function's name is looked up **by searching for it** in the global scope.

Change paragraph 26:

- If the *new-expression* **does not** begin with a unary `::` operator, the deallocation function's name is looked up in the global scope. Otherwise, if **and** the allocated type is a class type `T` or an array thereof, **a search is performed for the** deallocation function's name **is looked up** in the scope of `T`. **Otherwise, or if this lookup fails to find the name, or if the allocated type is not a class type or array thereof nothing is found**, the deallocation function's name is looked up **by searching for it** in the global scope.

[expr.delete]

Change paragraph 9 (incorporating some of [class.free]/4):

- ~~When~~**If** the keyword `delete` in a *delete-expression* is **not** preceded by the unary `::` operator, the deallocation function's name is looked up in global scope. Otherwise, the lookup considers class-specific deallocation functions (`{class.free}`); **and the type of the operand is a pointer to a (possibly cv-qualified) class type `T`:**
 1. **If `T` has a virtual destructor, the deallocation function is the one selected at the point of definition of the dynamic type's virtual destructor (`{class.dtor}`).**
 2. **Otherwise, a search is performed for the deallocation function's name in the scope of `T`.****Otherwise, or if no class-specific deallocation function** ~~thing~~ **is found**, the deallocation function's name is looked up **by searching for it** in the global scope. **In any case, any declarations other than of usual deallocation functions (`{basic.stc.dynamic.deallocation}`) are discarded. [Note: If only a placement deallocation function is found in a class, the program is ill-formed because the lookup set is empty (`{basic.lookup}`). — end note]**

Change paragraph 10:

- If ~~deallocation function lookup finds~~ more than one ~~usual~~ deallocation function **is found**, the function to be called is selected as follows:
 1. If any of the deallocation functions is a destroying operator `delete`, all deallocation functions that are not destroying operator deletes are eliminated from further consideration.
 2. If the type has new-extended alignment, a function with a parameter of type `std::align_val_t` is preferred; otherwise a function without such a parameter is preferred. If any preferred functions are found, all non-preferred functions are eliminated from further consideration.
 3. If exactly one function remains, that function is selected and the selection process terminates.
 4. If the deallocation functions ~~have~~**belong to a** class scope, the one without a parameter of type `std::size_t` is selected.
 5. If the type is complete and if, for an array delete expression only, the operand is a pointer to a class type with a non-trivial destructor or a (possibly multi-dimensional) array thereof, the function with a parameter of type `std::size_t` is selected.
 6. Otherwise, it is unspecified whether a deallocation function with a parameter of type `std::size_t` is selected.

[expr.const]

Change paragraph 12:

- An expression or conversion is in an *immediate function context* if it is potentially evaluated and its innermost **enclosing** non-block scope is a function parameter scope of an immediate function. An expression or conversion is an *immediate invocation* if it is an explicit or implicit invocation of an immediate function and is not in an immediate function context. An immediate invocation shall be a constant expression.

Change bullet (14.7):

- a variable ~~whose name appears as~~**named by** a potentially constant evaluated expression that is either a `constexpr` variable or is of non-volatile `const`-qualified integral type or of reference type.

[stmt.stmt]

[stmt.pre]

Remove paragraph 5.

[stmt.label]

Change paragraph 1:

- [...]

The optional *attribute-specifier-seq* appertains to the label. An *identifier label* declares the identifier. The only use of an identifier label with an *identifier* is as the target of a goto. The scope of a label is the function in which it appears. No two labels shall not be redeclared within a function shall have the same *identifier*. A label can be used in a goto statement that appears before its declaration. Labels have their own name space and do not interfere with other identifiers. [Note: A label may have the same name as another declaration in the same scope or a *template parameter* from an enclosing scope. Unqualified name lookup ([basic.lookup.unqual]) ignores labels. — end note]

[stmt.select]

Change paragraph 2:

- The substatement in a [Note: Each selection-statement (and each substatement, in of the else form of the if statement) implicitly defines has a block scope ([basic.scope.block]). — end note] If the substatement in a selection-statement is a single statement and not a compound-statement, it is as if it was rewritten to be a compound-statement containing the original substatement. [Example:

[...]

can be equivalently rewritten as

[...]

Thus after the if-statement, i is no longer in scope. — end example]

[stmt.if]

Change paragraph 3:

- [...]

except that names declared in the *init-statement* are is in the same declarative region scope as those declared in the condition.

[stmt.switch]

Change paragraph 7 as for [stmt.if]/3.

[stmt.iter]

Remove paragraph 3.

[stmt.while]

Change paragraph 2:

- When the condition of a while-statement is a declaration, the scope of the variable that is declared extends from its point of declaration ([basic.scope.pdecl]) to the end of the while-statement. A while statement is equivalent to

[...]

[Note: The variable created in the condition is destroyed and created with each iteration of the loop. [Example:

[...]

In the while-loop, the constructor and destructor are each called twice, once for the condition that succeeds and once for the condition that fails. — *end example*] — *end note*]

[stmt.for]

Change paragraph 1 as for [stmt.if]/3 (up to a comma instead of a period).

Remove paragraph 3 (whose example is moved to [basic.scope.block]).

[stmt.ranged]

Change bullet (1.3.2):

- if the *for-range-initializer* is an expression of class type *C*, the *unqualified-ids* and searches in the scope of *C* for the *names* *begin* and *end* are looked up in the scope of *C* as if by class member access lookup ([basic.lookup.classref]), and if both *each* find at least one declaration, *begin-expr* and *end-expr* are *range.begin()* and *range.end()*, respectively;

[stmt.dcl]

Change paragraph 1:

- [...]

[*Note:* If an identifier introduced by a declaration was previously declared in an outer block, the outer declaration is hidden for the remainder of the block. ([basic.lookup.unqual]), after which it resumes its force. — *end note*]

[stmt.ambig]

Change paragraph 3:

- [...] Disambiguation precedes parsing, and a statement disambiguated as a declaration may be an ill-formed declaration. If, during parsing, *lookup finds that* a name in a template *parameter is bound differently than it would be bound during a trial parse* *argument is bound to (part of) the declaration being parsed*, the program is ill-formed. No diagnostic is required. [*Note:* This can occur only when the name is declared earlier in the declaration. — *end note*]
[*Example:*

[...]

— *end example*]

[dcl.dcl]

[dcl.pre]

Change paragraph 4:

- A declaration occurs in a scope ([basic.scope]); the scope rules are summarized in [basic.lookup]. A *Certain* declarations that declares a function or defines a class, namespace, template, or function also has *contain* one or more scopes nested within it ([basic.scope]). These nested scopes, in turn, can have declarations nested within them. Unless otherwise stated, utterances in [dcl.dcl] about components in, of, or contained by a declaration or subcomponent thereof refer only to those components of the declaration that are *not* nested within scopes nested within the declaration.

Change paragraph 5:

- In a *simple-declaration*, the optional *init-declarator-list* can be omitted only when declaring a class ([class.pre]) or enumeration ([dcl.enum]), that is, when the *decl-specifier-seq* contains either a *class-specifier*, an *elaborated-type-specifier* with a *class-key* ([class.name]), or an *enum-specifier*. In these cases and whenever a *class-specifier* or *enum-specifier* is present in the *decl-specifier-seq*, the identifiers in these specifiers are *among the names being also* declared

by the declaration (as *class-names*, *enum-names*, or *enumerators*, depending on the syntax). In such cases, the *decl-specifier-seq* shall introduce one or more names into the program, or shall redeclare a name introduced by a previous declaration. [Example:

[...]

— end example]

Change paragraph 9:

- Each *init-declarator* in the *init-declarator-list* contains exactly one *declarator-id*, which is the name, the declared by that *init-declarator* and (hence) one of the names declared by the declaration introduce that name. [Note: Otherwise, the *declarator-id* is a *qualified-id* or its *unqualified-id* is a *template-id* and no name is introduced. — end note] The *defining-type-specifiers* ([*dcl.type*]) in the *decl-specifier-seq* and the recursive *declarator* structure of the *init-declarator* describe a type ([*dcl.meaning*]), which is then associated with the name being declared by the *init-declarator* *declarator-id*.

Change paragraph 10:

- If the *decl-specifier-seq* contains the *typedef* specifier, the declaration is called a *typedef declaration*, each *declarator-id* must be an *identifier*, and the name of each *init-declarator* is declared to be a *typedef-name*, synonymous with its associated type ([*dcl.typedef*]). If the *decl-specifier-seq* contains no *typedef* specifier, the declaration is called a *function declaration* if the type associated with the name a *declarator-id* is a function type ([*dcl.fct*]) and an *object declaration* otherwise.

[dcl.spec]

[dcl.stc]

Change paragraph 6:

- The linkages implied by successive All declarations for a given entity shall agree give its name the same linkage. That is, within a given scope, each declaration declaring the same variable name or the same overloading of a function name shall imply the same linkage. [Note: The linkage given by some declarations is affected by previous declarations. Overloaded functions are distinct entities. — end note] [Example:

[...]

— end example]

[dcl.typedef]

Change paragraph 2:

- A *typedef-name* can also be introduced by an *alias-declaration*. The *identifier* following the using keyword is not looked up; it becomes a *typedef-name* and the optional *attribute-specifier-seq* following the *identifier* appertains to that *typedef-name*. [...]

Remove paragraphs 3 through 7.

[dcl.type]

[dcl.type.elab]

Change paragraph 1:

- An *attribute-specifier-seq* shall not appear in an *elaborated-type-specifier* unless the latter is the sole constituent of a declaration. If an *elaborated-type-specifier* is the sole constituent of a declaration, the declaration is ill-formed unless it is an explicit specialization ([*temp.expl.spec*]), an explicit instantiation ([*temp.explicit*]) or it has one of the following forms:

```

class-key attribute-specifier-seqopt identifier ;
class-key attribute-specifier-seqopt simple-template-id ;
friend class-key ::opt identifier ;
friend class-key ::opt simple-template-id ;
friend class-key nested-name-specifier identifier ;
friend class-key nested-name-specifier templateopt simple-template-id ;

```

In the first case, it declares the *identifier* as a *class-name*. The second case shall appear only in a *template-declaration*; it declares a *partial specialization* ([temp.class.spec]). The *attribute-specifier-seq*, if any, appertains to the class or *template* being declared; the attributes in the *attribute-specifier-seq* are thereafter considered attributes of the class whenever it is named.

Insert before paragraph 2:

- Otherwise, the *elaborated-type-specifier* shall not have an *attribute-specifier-seq*. If it contains a *identifier* but no *nested-name-specifier* and (unqualified) lookup for the *identifier* finds nothing, it shall not be introduced by the enum keyword and declares the *identifier* as a *class-name*. Its target scope is the nearest enclosing namespace or block scope. If the declaration includes the friend specifier,

1. lookup for the *identifier* does not consider scopes that contain that scope, and
2. no name is bound.

[Note: [basic.lookup.elab] describes how name lookup proceeds in an *elaborated-type-specifier*. — end note]

- If it appears with the friend specifier as an entire declaration, the declaration shall have one of the following forms:

```

friend class-key nested-name-specifieropt identifier ;
friend class-key simple-template-id ;
friend class-key nested-name-specifier templateopt simple-template-id ;

```

Change paragraph 2:

- [Note: [basic.lookup.elab] describes how name lookup proceeds for the *identifier* in a *template-declaration*. An *elaborated-type-specifier* may be used to refer to a previously declared *class-name* or *enum-name* even though the name has been hidden by a non-type declaration. — end note] If the *identifier* or *simple-template-id* resolves to a *class-name* or *enum-name*, the *elaborated-type-specifier* introduces it into the declaration the same way a *simple-type-specifier* introduces its *type-name* ([dcl.type.simple]). [...]

[dcl.spec.auto]

Change paragraph 11:

- If the name of a *primary-expression* is an *id-expression* that refers to an entity with an undeduced placeholder type appears in an expression, the program is ill-formed. Once a non-discarded return statement has been seen in a function, however, the return type deduced from that statement can be used in the rest of the function, including in other return statements. [Example:

[...]

— end example]

[dcl.decl]

Change paragraph 2:

- The three components of a *simple-declaration* are the attributes ([dcl.attr]), the specifiers (*decl-specifier-seq*; [dcl.spec]) and the declarators (*init-declarator-list*). The specifiers indicate the type, storage class or other properties of the entities being declared. The declarators specify the names of these entities or their names and (optionally) modify the type of the specifiers with operators such as * (pointer to) and () (function returning). Initial values can also be specified in a declarator; initializers are discussed in [dcl.init] and [class.init].

[dcl.meaning]

Change paragraph 1:

- A declarator contains exactly one *declarator-id*; it names the *identifier-entity* that is declared. An *unqualified-id* occurring in a *declarator-id* shall be a simple *identifier* except for the declaration of some special functions ([*class.ctor*], [*class.conv*], [*class.dtor*], [*over.oper*]) and for the declaration of template specializations or, partial specializations, or explicit instantiations ([*temp.spec*]). ~~When the *declarator-id* is qualified, the declaration shall refer to a previously declared member of the class or namespace to which the qualifier refers (or, in the case of a namespace, of an element of the inline namespace set of that namespace ([*namespace.def*])) or to a specialization thereof; the member shall not merely have been introduced by a *using-declaration* in the scope of the class or namespace nominated by the *nested-name-specifier* of the *declarator-id*. The *nested-name-specifier* of a qualified *declarator-id* shall not begin with a *decltype-specifier*. [Note: If the qualifier is the global `::` scope resolution operator, the *declarator-id* refers to a name declared in the global namespace scope. — end note]~~ The optional *attribute-specifier-seq* following a *declarator-id* appertains to the entity that is declared.

Insert before paragraph 2, including the example from [*namespace.memdef*]/2:

- If the *id-expression* in a *declarator-id* is a *qualified-id* or *template-id*, or the declaration appears in an *explicit-instantiation* ([*temp.explicit*]), let *S* be the lookup context of the *qualified-id* ([*basic.lookup.qual*]), or the smallest enclosing namespace scope if the *id-expression* is a *template-id*. The declaration, taken as inhabiting *S* and having only the *unqualified-id* in the *declarator-id*, shall

1. if the *unqualified-id* is a *template-id* and the declaration is a friend declaration ([*class.friend*]), declare a specialization of an entity found by qualified name lookup for the name that is or is in the *unqualified-id* in *S*, or else
2. redeclare or declare a specialization of an entity that belongs to *S*, or if not that to an element of its inline namespace set if *S* is a namespace ([*namespace.def*]).

Only entities declared by a declaration that is reachable from the *declarator-id* and does not inhabit a block scope are considered. [Note: The previous declaration might be a friend declaration. — end note] If there is more than one entity that might be redeclared or specialized, the program is ill-formed. If the declaration is not a friend declaration, it shall inhabit a namespace that contains the entity. The *declarator*'s target scope is *S*; it binds no name. [Example:

[...]

— end example]

[Drafting note: Richard proposed making the restriction on use of a *qualified-id* grammatical. — end drafting note]

(The example below is moved from [*basic.link*]/8, with changes as marked.)

- If the *id-expression* in a *declarator-id* is a name, the name is not looked up.

1. If the declaration is a friend declaration, its target scope is the smallest enclosing namespace or block scope; in the latter case, the declaration shall redeclare a function with a reachable declaration inhabiting that scope. The *declarator* binds no name.
2. If the declaration is declared in a block scope *S* and declares a function ([*dcl.fct*]) or uses the *extern* specifier, the declaration shall not be attached to a named module ([*module.unit*]) and its target scope is the smallest enclosing namespace scope, but the name is bound in *S*. [Example:

```
namespace X {
    void p() {
        q();                      // error: q not yet declared
        extern void q();           // q is a member of namespace X
        extern void r();           // r is a member of namespace X
    }

    void middle() {
        q();                      // error: q not yet-declaredfound
    }

    void q() { /* ... */ }         // definition of X::q
}

void q() { /* ... */ }           // some other, unrelated q
void X::r() { /* ... */ }        // error: r cannot be declared by qualified-id
```

— end example]

Change paragraph 3:

- Thus, **(for each *init-declarator*)** a declaration **of a particular identifier** has the form

$T\ D$

where T is of the form *attribute-specifier-seq_{opt} decl-specifier-seq* and D is a declarator. Following is a recursive procedure for determining the type specified for the contained *declarator-id* by such a declaration.

Change paragraph 5:

- In a declaration *attribute-specifier-seq_{opt} T D* where D is an unadorned **identifier****name** the type of this identifier is “ T ”.

[dcl.fct]

Change paragraph 15:

- An identifier can optionally be provided as a parameter name; if present in a function definition ([dcl.fct.def]), it names a parameter. [Note: In particular, parameter names are also optional in function definitions and names used for a parameter in different declarations and the definition of a function need not be the same. — end note]

[dcl.fct.default]

Change paragraph 5:

- The default argument has the same semantic constraints as the initializer in a declaration of a variable of the parameter type, using the copy-initialization semantics ([dcl.init]). The names in the default argument are **bound****looked up**, and the semantic constraints are checked, at the point where the default argument appears. Name lookup and checking of semantic constraints for default arguments in function templates and in member functions of class templates are performed as described in [temp.inst]. [Example: In the following code, g will be called with the value $f(2)$:

[...]

— end example] [Note: **In member function declarations, names in default arguments are looked up as described in [basic.lookup.unqual].** Access checking applies to names in default arguments as described in [class.access]. — end note]

Change paragraph 9:

- A default argument is evaluated each time the function is called with no argument for the corresponding parameter. A parameter shall not appear as a potentially-evaluated expression in a default argument. [Note: Parameters of a function declared before a default argument are in scope and can hide namespace and class member names. — end note]

[Example:

[...]

— end example] A non-static member shall not appear in a default argument unless it appears as the *id-expression* of a class member access expression ([expr.ref]) or unless it is used to form a pointer to member ([expr.unary.op]).

[Example: The declaration of $x::mem1()$ in the following example is ill-formed because no object is supplied for the non-static member $x::a$ used as an initializer.

[...]

The declaration of $x::mem2()$ is meaningful, however, since no object is needed to access the static member $x::b$. Classes, objects, and members are described in [class.pre]. — end example] A default argument is not part of the type of a function. [Example:

[...]

— end example] When a declaration of a function **in a scope S is introduced****named by way of a using-declaration or** ([namespace.udecl]), any default argument **information associated with the****any** declaration **is made known as well** **D in S is available to any call from which D is reachable.** If the function is redeclared thereafter in the namespace with additional default arguments, the additional arguments are also known at any point following the

redeclaration where the *using-declaration* is in scope. [Note: *D* need not be reachable from the *using-declaration*. — end note]

[dcl.init]

Remove paragraph 13.

[dcl.init.aggr]

Change bullet (4.1):

- If the element is an anonymous union **objectvariable** and the initializer list is a *designated-initializer-list*, the anonymous union **objectvariable** is initialized by the *designated-initializer-list* $\{D\}$, where *D* is the *designated-initializer-clause* naming a member of the anonymous union **objectvariable**. [...]

[dcl.fct.def]

[dcl.fct.def.general]

Change paragraph 7:

- In the *function-body*, a **function-local predefined variable** denotes a block-scope object of **is a variable with** static storage duration that is implicitly defined (see [basic.scope.block]) **in the function parameter scope that contains each function-body**.

[dcl.fct.def.coroutine]

Change paragraph 10:

- **If a search for the unqualified-id name** `get_return_object_on_allocation_failure` **is looked up** in the scope of the promise type **by class member access lookup** ([basic.class.member.lookup.classref]). **If finds** any declarations **are found**, then the result of a call to an allocation function used to obtain storage for the coroutine state is assumed to return `nullptr` if it fails to obtain storage, and if a global allocation function is selected, the `::operator new(size_t, nothrow_t)` form is used. [...]

[dcl.struct.bind]

Change paragraph 4:

- Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete class type with a member named `value`, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression. Let *i* be an index prvalue of type `std::size_t` corresponding to v_i . **If a search for the unqualified-id name** `get` **is looked up** in the scope of *E* **by class member access lookup** ([basic.class.member.lookup.classref]), and if that finds at least one declaration that is a function template whose first template parameter is a non-type parameter, the initializer is `e.get<i>()`. Otherwise, the initializer is `get<i>(e)`, where `get` **is undergoes argument-dependent** **looked up** **in the associated namespaces** ([basic.lookup.argdep]). In either case, `get<i>` is interpreted as a *template-id*. [Note: Ordinary unqualified lookup ([basic.lookup.unqual]) is not performed. — end note][...]

[enum]

[dcl.enum]

Change paragraph 4:

- If an *enum-head-name* contains a *nested-name-specifier*, it shall not begin with a *decltype-specifier* and **the enclosing enum-specifier or opaque enum declaration shall refer to an** **there shall be a preceding declaration of the** enumeration that **was previously declared directly in** **inhabits the scope of** the class (template) or namespace to which the *nested-name-specifier* refers, or in an element of the inline namespace set ([namespace.def]) of that namespace **(i.e., neither**

inherited nor introduced by a *using-declaration*), and the *enum-specifier* or *opaque-enum-declaration* shall appear in a namespace *scope* enclosing the previous declaration.

Change paragraph 6:

- An enumeration whose underlying type is fixed is an incomplete type *from its point of declaration* ([*basic.scope.pdecl*]) to *until* immediately after its *enum-base* (if any), at which point it becomes a complete type. An enumeration whose underlying type is not fixed is an incomplete type *from its point of declaration to immediately after until* the closing } of its *enum-specifier*, at which point it becomes a complete type.

Change paragraph 11:

- *Each enum-name* and *The name of* each unscoped *enumerator* *enumerator* is declared *also bound* in the scope that immediately contains the *enum-specifier*. *Each scoped enumerator is declared in the scope of the enumeration. These names obey the scope rules defined for all names in [basic.scope] and [basic.lookup].* [Example:

[...]

— end example] *An enumerator declared in class scope can be referred to using the class member access operators (::, . (dot) and -> (arrow)); see [expr.ref].* [Example:

[...]

— end example]

[enum.udcl]

Change paragraph 2:

- A *using-enum-declaration* *introduces the enumerator names of the named enumeration as if by* *is equivalent to* a *using-declaration* for each enumerator.

Change paragraph 3:

- [Note: A *using-enum-declaration* in class scope *adds makes* the enumerators of the named enumeration *as members to the scope. This means they are accessible for available via* member lookup. [Example:

[...]

— end example] — end note]

[basic.namespace]

Change paragraph 1:

- A namespace is an optionally-named *declarative region* *entity whose scope can contain declarations of any kind of entity*. The name of a namespace can be used to access entities *declared in that belong to* that namespace; that is, the *members* *members* of the namespace. Unlike other *declarative regions* *entities*, the definition of a namespace can be split over several parts of one or more translation units *and modules*.

Change paragraph 2:

- [Note: A *namespace name with external linkage* *namespace-definition* is exported if *any of its namespace definitions* *it* is exported; or *if it* contains any *export-declarations* ([*module.interface*]). A namespace is never attached to a module, and *its name* never has module linkage *even if it is not exported*. — end note] [Example:

[...]

— end example]

Change paragraph 3:

- The *outermost declarative region of a translation unit* *re* is a *namespace* *global namespace with no name and no declaration*; see [*basic.scope.namespace*].

[namespace.def]

Change paragraph 1:

- Every *namespace-definition* shall appear ~~in the global scope or~~ in a namespace scope ([basic.scope.namespace]).

Change paragraph 7:

- Members of an inline namespace can be used in most respects as though they were members of the enclosing namespace. Specifically, ~~the inline namespace and~~ its enclosing namespace ~~are both added to the set of~~ is an associated namespaces ~~used in argument-dependent lookup ([basic.lookup.argdep]) whenever one of them is~~ if a member is an associated entity, and a *using-directive* ([namespace.udir]) that names the inline namespace is implicitly inserted into the enclosing namespace as for an unnamed namespace ([namespace.unnamed]). Furthermore, each member of the inline namespace can subsequently be partially specialized ([temp.class.spec]), explicitly instantiated ([temp.explicit]), or explicitly specialized ([temp.explicit.spec]) as though it were a member of the enclosing namespace. Finally, looking up a name in the enclosing namespace via explicit qualification ([namespace.qual]) will include members of the inline namespace ~~brought in by the using-directive~~ even if there are declarations of that name in the enclosing namespace.

Change paragraph 8:

- These properties are transitive: if a namespace *N* contains an inline namespace *M*, which in turn contains an inline namespace *O*, then the members of *O* can be used as though they were members of *M* or *N*. The *inline namespace set* of *N* is the transitive closure of all inline namespaces in *N*. ~~The enclosing namespace set of *O* is the set of namespaces consisting of the innermost non-inline namespace enclosing an inline namespace *O*, together with any intervening inline namespaces.~~

[namespace.memdef]

Remove subclause, referring the stable name to [namespace.def]. Remove the cross references in [namespace.def]/4 and [class.compare.default]/3, and update those in [temp.class.spec]/6 and [temp.explicit.spec]/3 to refer to [dcl.meaning].

The examples in paragraphs 2 and 3 are moved to [dcl.meaning] and [class.friend]/11 respectively.

[namespace.udir]

Remove paragraph 2.

Change paragraph 3:

- A *using-directive* does not ~~add any members to the declarative region in which it appears~~ bind any names. [Note: Instead, unqualified and qualified lookup search the namespace nominated by a *using-directive* (including for further *using-directives*) when considering certain scopes that contain it ([basic.lookup.unqual], [basic.lookup.qual]). — end note] [Example:

[...]

— end example]

Remove paragraphs 4 and 5.

Change paragraph 7:

- ~~During overload resolution, all functions from the transitive search are considered for argument matching. The set of declarations found by the transitive search is unordered. [Note: In particular, the order in which namespaces are considered and the relationships among the namespaces implied by the *using-directives* do not cause preference to be given to any of the declarations found by the search affect overload resolution.~~ — end note] An ambiguity exists if the best match finds two functions with Neither is any function excluded because another has the same signature, even if one is in a namespace reachable through *using-directives* in the namespace of the other.^[...] — end note] [Example:

[...]

— end example]

[namespace.udcl]

Change and split paragraph 1:

- Each *using-declarator* in a *using-declaration*^[...] introduces a set of declarations into the declarative region in which the *using-declaration* appears. The names the set of declarations introduced by the *using-declarator* is found by performing qualified name lookup ([basic.lookup.qual], [class.member.lookup]) for the name in the *using-declarator*, excluding except that class and enumeration declarations that would be discarded are merely ignored when checking for ambiguity ([basic.lookup]), conversion function templates are ignored, and certain functions that are hidden as described below. If the *using-declarator* is a qualified dependent name ([temp.dep.type]), it is considered to name a constructor if and only if the *unqualified-id* is the same as the terminal name of the *nested-name-specifier*. If the lookup in any instantiation finds that a *using-declarator* that is not considered to name a constructor does so, or that a *using-declarator* that is considered to name a constructor does not, the program is ill-formed.
- If the *using-declarator* does not name a constructor, the *unqualified-id* is declared in the declarative region in which the *using-declaration* appears as a synonym for each declaration introduced by bound to the *using-declarator*. [Note: Only the specified name is so declared bound; specifying an enumeration name in a *using-declaration* does not declare bind the names of its enumerators in the *using-declaration*'s declarative region. During name lookup, the *using-declarator* is replaced with the declarations it names ([basic.lookup]). — end note] If the *using-declarator* names a constructor, it declares that the class inherits the named set of constructor declarations introduced by the *using-declarator* from the nominated base class. [Note: For the keyword typename, see [temp.res]. — end note]

Remove paragraph 2.

Change paragraph 3:

- In a *using-declaration* used as a *member-declaration*, each *using-declarator* shall either name an enumerator or have a *nested-name-specifier* naming a base class of the innermost enclosing class being defined. [Example:

[...]

— end example] If a *using-declarator* names a constructor, its *nested-name-specifier* shall name a direct base class of the innermost enclosing class being defined. If the immediate (class) scope is associated with a class template, it shall derive from the specified base class or have at least one dependent base class. [Example:

[...]

— end example] [Example:

[...]

— end example]

Change paragraph 4:

- [Note: Since destructors do not have names, a *using-declaration* cannot refer to a destructor for a base class. Since specializations of member templates for conversion functions are not found by name lookup, they are not considered when a *using-declaration* specifies a conversion function ([temp.mem]). — end note] If a constructor or assignment operator brought from a base class into a derived class has the signature of a copy/move constructor or assignment operator for the derived class ([class.copy.ctor], [class.copy.assign]), the *using-declaration* does not by itself suppress the implicit declaration of the derived class member; the member from the base class is hidden or overridden by the implicitly-declared copy/move constructor or assignment operator of the derived class, as described below.

Remove paragraph 8.

Replace paragraph 9:

- If a declaration is named by two *using-declarators* that inhabit the same class scope, the program is ill-formed.

Change paragraph 10:

- [Note: For a *using-declaration* whose *nested-name-specifier* names a namespace, members does not name declarations added to the namespace after the *using-declaration* are not in the set of introduced declarations, so they are not considered when a use of the name is made it. Thus, additional overloads added after the *using-declaration* are

ignored, but default function arguments ([dcl.fct.default]), default template arguments ([temp.param]), and template specializations ([temp.class.spec], [temp.expl.spec]) are considered. — end note] [Example:

[...]

— end example]

Change paragraph 12:

- Since a *using-declaration* is a declaration, the restrictions on declarations of the same name in the same declarative region ([basic.scope]) also apply to *using-declarations*. If a declaration named by a *using-declaration* that inhabits the target scope of another declaration shall potentially conflict with it and either is reachable from the other, the program is ill-formed. If two declarations named by *using-declarations* that inhabit the same scope potentially conflict, either is reachable from the other, and they do not both declare functions or function templates, the program is ill-formed. [Note: Overload resolution may not be able to distinguish between conflicting function declarations. — end note]

[Example:

```
namespace A {
    int x;
    int f(int);
    int g;
    void h();
}

namespace B {
    int i;
    struct g { };
    struct x { };
    void f(int);
    void f(double);
    void g(char);    // OK: hides struct g
}

void func() {
    int i;
    using B::i;      // error: i declared twiceconflicts
    void f(char);
    using B::f;      // OK: each f is a function
    using A::f;      // OK, but merely prevents calling B::f(int).
    f(1);            // error: ambiguous
    f(3.5);          // calls B::f(double)
    using B::g;
    g('a');          // calls B::g(char)
    struct g g1;      // g1 has class type B::g
    using A::g;      // error: conflicts with B::g
    void h();
    using A::h;      // error: conflicts
    using B::x;
    using A::x;      // OK: hides struct B::x
    x = 99;          // assigns to A::x
    struct x x1;      // x1 has class type B::x
}
```

— end example]

Remove paragraph 13 (whose example is merged into the above).

Change paragraph 14:

- WhenThe set of declarations named by a *using-declarator* brings declarations from a base class into a derived class, that inhabits a class C does not include member functions and member function templates in the derived of a base class override and/or hidefor which C declares member functions and member function templates with the same name, parameter-type-list ([dcl.fct]), trailing *requires-clause* (if any), cv-qualification, and *ref-qualifier* (if any), in a base class (rather than conflicting). Such hidden or overridden declarations are excluded from the set of declarations introduced by the *using-declarator*. [Example:

[...]

— end example]

Change paragraph 16:

- Constructors that are **introduced** by a *using-declaration* are treated as though they were constructors of the derived class when looking up the constructors of the derived class ([class.qual]) or forming a set of overload candidates ([over.match.ctor], [over.match.copy], [over.match.list]). [Note: If such a constructor is selected to perform the initialization of an object of class type, all subobjects other than the base class from which the constructor originated are implicitly initialized ([class.inhctor.init]). A constructor of a derived class is sometimes preferred to a constructor of a base class if they would otherwise be ambiguous ([over.match.best]). — end note]

Change paragraph 17:

- In a *using-declarator* that does not name a constructor, **all members of the set of introduced** declarations **named** shall be accessible. In a *using-declarator* that names a constructor, no access check is performed. **In particular, if a derived class uses a using-declarator to access a member of a base class, the member name shall be accessible. If the name is that of an overloaded member function, then all functions named shall be accessible. The** Every base class members **mentioned** by a *using-declarator* shall be **visible in the scope** **a result of a search for the terminal name of the using-declarator in** at least one of the direct base classes of the class **where** **which** the *using-declarator* is **specified** **inhabits**.

Change paragraph 19:

- A **synonym created by a** *using-declaration* has the usual accessibility for a *member-declaration*. **A** **Base-class constructors considered because of a** *using-declarator* **that names a constructor does not create a synonym; instead, the additional constructors** are accessible if they would be accessible when used to construct an object of the **corresponding** base class, **and**; the accessibility of the *using-declaration* is ignored. [Example:

[...]

— end example]

Remove paragraph 21.

[dcl.link]

Change paragraph 4:

- Linkage specifications nest. When linkage specifications nest, the innermost one determines the language linkage. [Note: A linkage specification does not establish a scope. — end note] A *linkage-specification* shall **occur only** **inhabit a** namespace scope ([basic.scope]). In a *linkage-specification*, the specified language linkage applies to the function types of all function declarators, function names with external linkage, and variable names with external linkage declared within the *linkage-specification*. [Example:

[...]

— end example] A C language linkage is ignored in determining the language linkage of the names of class members and the function type of class member functions. [Example:

[...]

— end example]

Change paragraph 5:

- If two declarations declare functions with the same name and parameter-type-list ([dcl.fct]) to be members of the same namespace or declare objects with the same name to be members of the same namespace and the declarations give the names different language linkages, the program is ill-formed; no diagnostic is required if the declarations appear in different translation units. **Except for functions with C++ linkage, a function declaration without a linkage specification shall not precede the first linkage specification for that function. A redeclaration of a function can be declared without a linkage specification after an explicit linkage specification has been seen; the linkage explicitly specified in the earlier declaration is** **does** not affected by such a function declaration **its** **language linkage**.

[module]

[module.interface]

Change paragraph 1:

- An *export-declaration* shall ~~appear only at~~inhabit a namespace scope and ~~only appear~~ in the purview of a module interface unit. An *export-declaration* shall not appear directly or indirectly within an unnamed namespace or a *private-module-fragment*. An *export-declaration* has the declarative effects of its *declaration* or its *declaration-seq* (if any). ~~An export-declaration does not establish a scope and its~~The *declaration* or *declaration-seq* ~~of an export-declaration~~ shall not contain an *export-declaration*. [Note: An *export-declaration* does not establish a scope.]

[module.import]

Change paragraph 1:

- In a module unit, all *module-import-declarations* shall ~~precede~~appear before all other *top-level-declarations* in the *top-level-declaration-seq* of the *translation-unit* and of the *private-module-fragment* (if any). The optional *attribute-specifier-seq* appertains to the *module-import-declaration*.

Change paragraph 2:

- A *module-import-declaration* imports a set of translation units determined as described below. [Note: Namespace-scope names exported by the imported translation units ~~become visible~~can be found by name lookup ([basic.scope.namespacelookup]) in the importing translation unit and declarations within the imported translation units become reachable ([module.reach]) in the importing translation unit after the import declaration. — end note]

Change the footnote in paragraph 6:

- This is consistent with the lookup rules for ~~visibility of~~ imported names ([basic.scope.namespacelookup]).

[module.context]

Change paragraph 1:

- The *instantiation context* is a set of points within the program that determines which ~~names~~declarations are ~~visible~~to found by argument-dependent name lookup ([basic.lookup.argdep]) and which ~~declarations~~ are reachable ([module.reach]) in the context of a particular declaration or template instantiation.

[module.reach]

Change paragraph 1:

- A translation unit *U* is *necessarily reachable* from a point *P* if *U* is a module interface unit on which the translation unit containing *P* has an interface dependency, or the translation unit containing *P* imports *U*, in either case prior to *P* ([module.import]). [Note: While module interface units are reachable even when they are only transitively imported via a non-exported import declaration, namespace-scope names from such module interface units are not ~~visible~~to found by name lookup ([basic.scope.namespacelookup]). — end note]

Change paragraph 3:

- A declaration *D* is *reachable* ~~from~~ if, for any a point *P* in the instantiation context ([module.context]); if
 1. *D* appears prior to *P* in the same translation unit, or
 2. *D* is not discarded ([module.global.frag]), appears in a translation unit that is reachable from *P*, and either does not appear within a *private-module-fragment* or appears in a *private-module-fragment* of the module containing *P*.

A declaration is *reachable* if it is reachable from any point in the instantiation context ([module.context]). [Note: Whether a declaration is exported has no bearing on whether it is reachable. — end note]

Change paragraph 4:

- The accumulated properties of all reachable declarations of an entity within a context determine the behavior of the entity within that context. [Note: These reachable semantic properties include type completeness, type definitions, initializers, default arguments of functions or template declarations, attributes, ~~visibility of class or enumeration member names to ordinary lookup~~ **bound**, etc. Since default arguments are evaluated in the context of the call expression, the reachable semantic properties of the corresponding parameter types apply in that context. [Example:

[...]

— end example] — end note]

Change paragraph 5:

- [Note: An entity can have reachable declarations even if ~~it is not visible to~~ **no declaration of it can be found by** name lookup. — end note] [Example:

[...]

— end example]

[class]

[class.pre]

Change paragraph 1:

- [...]

A class declaration where the *class-name* in the *class-head-name* is a *simple-template-id* shall be an explicit specialization ([temp.expl.spec]) or a partial specialization ([temp.class.spec]). **Otherwise, the identifier is not looked up.** A *class-specifier* whose *class-head* omits the *class-head-name* defines an unnamed class. [Note: An unnamed class thus can't be final. — end note]

Change paragraph 2:

- A *class-name* is ~~inserted into the scope in which it is declared immediately after the class-name is seen. The class-name is also inserted into~~ **bound in** the scope of the class itself; this is known as the *injected-class-name*. [...]

Change paragraph 3:

- If a *class-head-name* contains a *nested-name-specifier*, ~~there shall be a preceding declaration of the class-specifier shall refer to a class that was previously declared directly in~~ **inhabits the scope of** the class **(template)** or namespace to which the *nested-name-specifier* refers, or in an element of the inline namespace set ([namespace.def]) of that namespace ~~(i.e., not merely inherited or introduced by a using-declaration)~~, and the *class-specifier* shall appear in a namespace **scope** enclosing the previous declaration. In such cases, the *nested-name-specifier* of the *class-head-name* of the definition shall not begin with a *decltype-specifier*.

[class.name]

Change paragraph 2:

- ~~A class declaration introduces the class name into the scope where it is declared and hides any class, variable, function, or other declaration of that name in an enclosing scope ([basic.scope]). If~~ **[Note: It may be necessary to use an elaborated-type-specifier to refer to** a class **name is declared in that belongs to** a scope **whereto which** a variable, function, or enumerator of the same name **is also declared, then when both declarations are in scope, the class can be referred to only using an elaborated-type-specifier** **belongs** ([basic.lookup.elab]). [Example:

[...]

— end example] — end note] **[Note: An elaborated-type-specifier declaration consisting solely of class-key identifier; is either a redeclaration of the name in the current scope or a forward declaration of the may also be used to declare an identifier as a class name. It introduces the class name into the current scope.** [Example:

[...]

— end example] — end note] [Note: Such declarations allow definition of classes that refer to each other. [Example: [...]

Declaration of friends is described in [class.friend], operator functions in [over.oper]. — end example] — end note]

[class.mem]

Change paragraph 1:

- [...] A *direct member* of a class *x* is a member of *x* that was first declared within the *member-specification* of *x*, including anonymous union ~~objects~~ **variables** ([class.union.anon]) and direct members thereof. [...]

Insert before paragraph 6:

- A *closed context* of a class (template) is a function body ([dcl.fct.def.general]) or *noexcept-specifier* ([except.spec]) within its *member-specification*.

Change paragraph 6:

- A *complete-class context* of a class **(template)** is **a closed context of the entity or** a
 1. ~~function body ([dcl.fct.def.general]),~~
 2. default argument ([dcl.fct.default]),
 3. ~~noexcept-specifier ([except.spec])~~ **default template argument ([temp.param]),** or
 4. default member initializer

within the *member-specification* of the class **or class template**. [Note: [...] — end note][...]

[class.mfct]

Change paragraph 1:

- ~~A~~ **If** a member function ~~may be~~ **is** defined ([dcl.fct.def]) in its class definition, ~~in which case~~ it is an inline member function ([dcl.inline]), ~~or it may be defined outside of its class definition if it has already been declared but not defined in its class definition. A member function definition that appears outside of the class definition shall appear in a namespace scope enclosing the class definition.~~ Except for member function definitions that appear outside of a class definition, and except for explicit specializations of member functions of class templates and member function templates ([temp.spec]) appearing outside of the class definition, a member function shall not be redeclared.

Remove paragraph 4 except for its example.

[class.mfct.non-static]

Change paragraph 3:

- When an *id-expression* ([expr.prim.id]) that is not part of a class member access syntax ([expr.ref]) and not used to form a pointer to member ([expr.unary.op]) is used ~~in a member of~~ **where the current** class ~~is~~ **x** ~~in a context where this can be used~~ ([expr.prim.this]), if name lookup ([basic.lookup]) resolves the name in the *id-expression* to a non-static non-type member of some class *c*, and if either the *id-expression* is potentially evaluated or *c* is *x* or a base class of *x*, the *id-expression* is transformed into a class member access expression ([expr.ref]) using (*this) ~~([class.this])~~ as the *postfix-expression* to the left of the . operator. [...]

Remove paragraph 4.

[class.this]

Remove subclause, referring the stable name to [expr.prim.this].

[class.ctor]

Change paragraph 2:

- A constructor is used to initialize objects of its class type. **[Note:** Because constructors do not have names, they are never found during **unqualified** name lookup; however an explicit type conversion using the functional notation ([expr.type.conv]) will cause a constructor to be called to initialize an object. **[Note:** The syntax looks like an explicit call of the constructor. — end note] **[Example:**

[...]

— end example] **[Note:** For initialization of objects of class type see [class.init]. — end note]

[class.conv]

Change paragraph 5:

- ~~User-defined conversions are used implicitly only if they are unambiguous.~~ **[Note:** A conversion function in a derived class does not hide a conversion function in a base class unless the two functions convert to the same type. Function overload resolution ([over.match.best]) selects the best conversion function to perform the conversion. — end note] **[Example:**

[...]

— end example]

[class.conv.fct]

Change paragraph 1:

- [...] specifies a conversion from x to the type specified by the *conversion-type-id*, **interpreted as a type-id ([dcl.name])**. [...]

[class.static]

[class.static.mfct]

Change paragraph 2:

- ~~[Note: A static member function does not have a this pointer ([class.expr.prim.this]). — end note] A static member function shall not be virtual. There shall not be a static and a non-static member function with the same name and the same parameter types ([over.load]). A static member function shall not be declared cannot be qualified with const, volatile, or const volatile virtual. — end note]~~

[class.static.data]

Change paragraph 3:

- The declaration of a non-inline static data member in its class definition is not a definition and may be of an incomplete type other than cv void. ~~The definition for a static data member that is not defined inline in the class definition shall appear in a namespace scope enclosing the member's class definition. In the definition at namespace scope, the name of the static data member shall be qualified by its class name using the :: operator. [Note: The initializer-expression in the definition of a static data member is in the scope of its class ([basic.scope.class]). — end note]~~ **[Example:**

[...] — end example]

[Note: [...] — end note]

[class.nest]

Change paragraph 1:

- A class can be declared within another class. A class declared within another is called a *nested class*. ~~The name of a nested class is local to its enclosing class. The nested class is in the scope of its enclosing class.~~ **[Note:** See [expr.prim.id] for restrictions on the use of non-static data members and non-static member functions. — end note]

[Example:

[...]

— end example]

Change paragraph 4:

- Like a member function, a friend function ([class.friend]) defined within a nested class is in the lexical scope of that class; it obeys the same rules for name binding as a static member function of that class ([class.static]), but it has no special access rights to members of an enclosing class.

[class.nested.type]

Remove subclause, referring the stable name to [diff.basic].

[class.union.anon]

Change paragraph 1:

- A union of the form

```
union { member-specification } ;
```

is called an *anonymous union*; it defines an unnamed type and an unnamed *object variable* of that type called an *anonymous union object variable*. Each *member-declaration* in the *member-specification* of an anonymous union shall either define *one or more* non-static data members, in which case each *member-declarator* is considered to be a *declaration of the named member of the anonymous union variable*, or be a *static_assert-declaration*. Nested types, anonymous unions, and functions shall not be declared within an anonymous union. The names of the members of an anonymous union are bound in the scope that encloses the union declaration and shall be distinct from the names of any other entity in the scope in which the anonymous union is declared. For the purpose of name lookup, after the anonymous union definition, the members of the anonymous union are considered to have been defined in the scope in which the anonymous union is declared. [Example:

[...]

Here a and p are used like ordinary (non-member) variables, but since they are union members they have the same address. — end example]

[class.local]

Change paragraph 1:

- A class can be declared within a function definition; such a class is called a *local class*. The name of a local class is local to its enclosing scope. The local class is in the scope of the enclosing scope, and has the same access to names outside the function as does the enclosing function. [Note: [...] — end note] [Example:

[...]

— end example]

[class.derived]

Change paragraph 2:

- A *class-or-decltype* shall denote a (possibly cv-qualified) class type that is not an incompletely defined class ([class.mem]); any cv-qualifiers are ignored. The class denoted by the *class-or-decltype* of a *base-specifier* is called a *direct base class* for the class being defined. During the lookup for a base class the name in the *type-name* or *simple-template-id*, non-type names are ignored ([basic.scope.hiding.lookup]). A class B is a base class of a class D if it is a direct base class of D or a direct base class of one of D's base classes. A class is an *indirect base class* of another if it is a base class but not a direct base class. A class is said to be (directly or indirectly) *derived* from its (direct or indirect) base classes. [Note: See [class.access] for the meaning of *access-specifier*. — end note] Unless redeclared in the derived class, members of a base class are also considered to be members of the derived class. Members of a base

class other than constructors are said to be *inherited* by the derived class and are also considered to be members of it. Constructors of a base class can also be inherited as described in [namespace.udecl]. **[Note:** Inherited members can be referred to in expressions in the same manner as other members of the derived class, unless their names are hidden or ambiguous ([class.member.lookup]). **Note:** The scope resolution operator `::` ([expr.prim.id.qual]) can be used to refer to a direct or indirect base member explicitly. This allows access to a name that has been redeclared in the derived class. A derived class can itself serve as a base class subject to access control; see [class.access.base]. A pointer to a derived class can be implicitly converted to a pointer to an accessible unambiguous base class ([conv.ptr]). An lvalue of a derived class type can be bound to a reference to an accessible unambiguous base class ([dcl.init.ref]). — *end note*]

[class.virtual]

Change paragraph 9:

If the class type in the covariant return type of `D::f` differs from that of `B::f`, the class type in the return type of `D::f` shall be complete at the pointlocus of the overriding declaration of `D::f` or shall be the class type `D`. When the overriding function is called as the final overrider of the overridden function, its result is converted to the type returned by the (statically chosen) overridden function ([expr.call]). *[Example: > [...] — end example]*

[class.access]

Change paragraph 1:

- A member of a class can be

1. private; that is, its name can be **usnamed** only by members and friends of the class in which it is declared.
2. protected; that is, its name can be **usnamed** only by members and friends of the class in which it is declared, by classes derived from that class, and by their friends (see [class.protected]).
3. public; that is, its name can be **usnamed** anywhere without access restriction.

[Note: A constructor or destructor can be named by an expression ([basic.def.odr]) even though it has no name. — *end note*]

Change paragraph 2:

- A member of a class can also access all the **name member**s to which the class has access. A local class of a member function may access the same **name member**s that the member function itself may access. [...]

Change paragraph 5:

- **[Note:** Access to members and base classes is controlled, not their visibility ([basic.scope.hiding]). Names of **does not prevent** members are still visible, and **from being found by name lookup or** implicit conversions to base classes **are still from being** considered, when those members and base classes are inaccessible. — *end note*] The interpretation of a given construct is established without regard to access control. If the interpretation established makes use of inaccessible member names or base classes, the construct is ill-formed.

Change paragraph 8:

- **Access is checked for** the names in a default argument ([dcl.fct.default]) **are bound** at the point of declaration, and **access is checked at that point** rather than at any points of use of the default argument. Access checking for default arguments in function templates and in member functions of class templates is performed as described in [temp.inst].

[class.friend]

Change paragraph 2:

- Declaring a class to be a friend implies that the names of private and protected members from the class granting friendship can be accessed in the *base-specifiers* and member declarations of the befriended class. *[Example:*

[...]

— *end example*] *[Example:*

[...]

— end example]

A class shall not be defined in a friend declaration. [Example:

[...]

— end example]

Change paragraph 5:

- When a [Note: A friend declaration refers to an overloaded entity, not (all overloads of) a name or operator, only the function specified by the parameter types becomes a friend. — end note] A member function of a class X can be a friend of a class Y. [Example:

[...]

— end example]

Change paragraph 6:

- A function can be defined in a friend declaration of a class if and only if the class is a non-local class ([class.local]); and the function name is unqualified, and the function has namespace scope. [Example:

[...]

— end example]

Change paragraph 7:

- Such a function is implicitly an inline function ([dcl.inline]). A [Note: If a friend function is defined in outside a class, it is not in the (lexical) scope of the class in which it is defined. A friend function defined outside the class is not ([basic.lookup.unqual]). — end note]

Replace paragraph 11, adding the example from [namespace.memdef]/3 and then retaining its example:

- [Note: A friend declaration never binds any names ([dcl.meaning], [dcl.type.elab]). — end note] [Example:

[...]

— end example] [Example:

[...]

— end example]

[class.paths]

Change paragraph 1:

- If a name declaration can be reached by several paths through a multiple inheritance graph, the access is that of the path that gives most access. [Example:

[...] — end example]

[class.init]

[class.base.init]

Change paragraph 2:

- If Lookup for an unqualified name in a mem-initializer-id an initial unqualified identifier is looked up in the scope of the constructor's class and, if not found in that scope, it is looked up in the scope containing the constructor's

definition ignores the constructor's function parameter scope. [Note: If the constructor's class contains a member with the same name as a direct or virtual base class of the class, a *mem-initializer-id* naming the member or base class and composed of a single identifier refers to the class member. A *mem-initializer-id* for the hidden base class may be specified using a qualified name. — end note] Unless the *mem-initializer-id* names the constructor's class, a non-static data member of the constructor's class, or a direct or virtual base of that class, the *mem-initializer* is ill-formed.

Change paragraph 15:

- Names in **t** [Note: The *expression-list* or *braced-init-list* of a *mem-initializer* are evaluated in the **function parameter** scope of the constructor for which the *mem-initializer* is specified, and can use **this** to refer to the object being initialized. — end note] [Example:

[...]

initializes `X::r` to refer to `X::a`, initializes `X::b` with the value of the constructor parameter `i`, initializes `X::i` with the value of the constructor parameter `i`, and initializes `X::j` with the value of `X::i`; this takes place each time an object of class `X` is created. — end example] [Note: Because the *mem-initializer* are evaluated in the scope of the constructor, the **this** pointer can be used in the *expression-list* of a *mem-initializer* to refer to the object being initialized. — end note]

[class.copy.elision]

Change bullet (1.2):

- in a *throw-expression* (`[expr.throw]`), when the operand is the name of a non-volatile automatic object (other than a function or catch-clause parameter) whose **that belongs to a** scope **that** does not **extend beyond the end of** **contain** the innermost enclosing **compound-statement associated with a** *try-block* (if there is one), the copy/move operation from the operand to the exception object (`[except.throw]`) can be omitted by constructing the automatic object directly into the exception object

Change paragraph 2:

- [Example:

[...]

Here the criteria for elision can eliminate the copying of the local automatic object `t` into the result object for the function call `f()`, which is the **global non-local** object `t2`. Effectively, the construction of **the local object** `t` can be viewed as directly initializing **the global object** `t2`, and that object's destruction will occur at program exit. Adding a move constructor to `Thing` has the same effect, but it is the move construction from the local automatic object to `t2` that is elided. — end example]

Change bullet (3.2):

- if the operand of a *throw-expression* (`[expr.throw]`) is a (possibly parenthesized) *id-expression* that names an implicitly movable entity whose **that belongs to a** scope **that** does not **extend beyond** **contain** the *compound-statement* of the innermost *try-block* or *function-try-block* (if any) whose *compound-statement* or *ctor-initializer* **encloses** **contains** the *throw-expression*,

[class.free]

Remove paragraph 3.

Remove paragraph 4, part of which is incorporated in `[expr.delete]` (*q.v.*).

Change paragraph 7:

- Access to the deallocation function is checked statically. **Hence**, even though a different one might actually be executed, **the statically visible deallocation function is required to be accessible**. [Example: For the call on line “// 1” above, if `B::operator delete()` had been private, the delete expression would have been ill-formed. — end example]

[over]

[over.pre]

Change paragraph 2:

- When a function **name** is **named** in a call, which function declaration is being referenced and the validity of the call are determined by comparing the types of the arguments at the point of use with the types of the parameters in the declarations **that are visible at the point of use in the overload set**. This function selection process is called *overload resolution* and is defined in [over.match]. [Example:

[...]

— end example]

[over.dcl]

Remove paragraph 2.

[over.match.funcs]

Insert before paragraph 7:

- In each case where conversion functions of a class *S* are considered for initializing an object or reference of type *T*, the candidate functions include the result of a search for the *conversion-function-id* operator *τ* in *S*. [Note: This search may find a specialization of a conversion function template ([basic.lookup]). — end note] Each case also defines sets of other permissible types (that are suitable for the initialization) for explicit and non-explicit conversion functions; each such conversion function that is a non-hidden member of *S* and yields a permissible type is a candidate function. If initializing an object, for any permissible type *cv U*, *cv U&* and any *cv2 U* are also permissible types. If the set of permissible types for explicit conversion functions is empty, any search results that are explicit are not candidates.

Change paragraph 7:

- In each case where a candidate is a function template, candidate function template specializations are generated using template argument deduction ([temp.over], [temp.deduct]). If a constructor template or conversion function template has an *explicit-specifier* whose *constant-expression* is value-dependent ([temp.dep]), template argument deduction is performed first and then, if the context **requires-admits only**, candidates that **is are** not explicit and the generated specialization is explicit ([dcl.fct.spec]), it will be removed from the candidate set. Those candidates are then handled as candidate functions in the usual way.^[...] A given name can refer to, **or a conversion can consider**, one or more function templates **and also to as well as** a set of non-template functions. In such a case, the candidate functions generated from each function template are combined with the set of non-template candidate functions.

[over.call.func]

Change paragraph 1:

- Of interest in [over.call.func] are only those function calls in which the *postfix-expression* ultimately contains a **named id-expression** that denotes one or more functions that might be called. [...]

Change paragraph 2:

- In qualified function calls, the **name to be resolved function** is **named by** an *id-expression* **and is** preceded by an *->* or *.* operator. Since the construct *A->B* is generally equivalent to *(*A).B*, the rest of [over.pre] assumes, without loss of generality, that all member function calls have been normalized to the form that uses an object and the *.* operator. Furthermore, [over.pre] assumes that the *postfix-expression* that is the left operand of the *.* operator has type “*cv T*” where *T* denotes a class.^[...] **Under this assumption, the id-expression in the call is looked up as a member function of T following the rules for looking up names in classes ([class.member.lookup]).** The function declarations found by **that name lookup ([class.member.lookup])** constitute the set of candidate functions. The argument list is the *expression-list* in the call augmented by the addition of the left operand of the *.* operator in the normalized member function call as the implied object argument ([over.match.funcs]).

Change paragraph 3:

- In unqualified function calls, the **function is named** is not qualified by an `→` or `→` operator and has the more general form of a *primary-expression*. The name is looked up in the context of the function call following the normal rules for name lookup in expressions ([basic.lookup]). The function declarations found by that name lookup ([basic.lookup]) constitute the set of candidate functions. Because of the rules for name lookup, the set of candidate functions consists (1) entirely of non-member functions or (2) entirely of member functions of some class T . In case (1), the argument list is the same as the *expression-list* in the call. In case (2), the argument list is the *expression-list* in the call augmented by the addition of an implied object argument as in a qualified function call. If **the current class is or is derived from T and the keyword `this` ([class.expr.prim.this]) is in scope and refers to class T , or a derived class of T** , then the implied object argument is `(*this)`. **If the keyword `this` is not in scope or refers to another class, then** Otherwise, a contrived object of type T becomes the implied object argument. **[...]**; **If the argument list is augmented by a contrived object and overload resolution selects one of the** non-static member functions of T , the call is ill-formed.

[over.call.object]

Change paragraph 1:

- If the *postfix-expression* E in the function call syntax evaluates to a class object of type “*cv T*”, then the set of candidate functions includes at least the function call operators of T . The function call operators of T are **obtained by ordinary lookup the results of a search for** the name operator `()` in the **context scope** of **$(E).operator()T$** .

[over.match.oper]

Change bullet (3.2):

- **For the operators `=`, `[]`, or `→`**, the set of non-member candidates is **empty; otherwise, it includes** the result of the unqualified lookup **of for operator@** in the context of the expression according to the usual rules for name lookup in **unqualified in the rewritten** function calls ([basic.lookup.argdep]) **except that, ignoring** all member functions are ignored. However, if no operand has a class type, only those non-member functions in the lookup set that have a first parameter of type T_1 or “reference to *cv T*1”, when T_1 is an enumeration type, or (if there is a right operand) a second parameter of type T_2 or “reference to *cv T*2”, when T_2 is an enumeration type, are candidate functions.

[over.match.copy]

Change bullet (1.2):

- When the type of the initializer expression is a class type “*cv S*”, **the non-explicit conversion functions of S and its base classes** are considered. **The permissible types for non-explicit conversion functions are T and any class derived from T** . When initializing a temporary object ([class.mem]) to be bound to the first parameter of a constructor where the parameter is of type “reference to possibly *cv*-qualified T ” and the constructor is called with a single argument in the context of direct-initialization of an object of type “*cv2 T*”, **the permissible types for** explicit conversion functions are **also considered the same; otherwise there are none**. Those that are not hidden within S and yield a type whose *cv*-unqualified version is the same type as T or is a derived class thereof are candidate functions. A call to a conversion function returning “reference to x ” is a glvalue of type x , and such a conversion function is therefore considered to yield x for this process of selecting candidate functions.

[over.match.conv]

Change paragraph 1:

- Under the conditions specified in [dcl.init], as part of an initialization of an object of non-class type, a conversion function can be invoked to convert an initializer expression of class type to the type of the object being initialized. Overload resolution is used to select the conversion function to be invoked. Assuming that “*cv1 T*” is the type of the object being initialized, and “*cv S*” is the type of the initializer expression, with S a class type, the candidate functions are selected as follows:

1. The **conversion functions of S and its base classes** are considered. Those **permissible types for** non-explicit conversion functions **that are not hidden within S and yield type T or a type U that can be converted to type T via a standard conversion sequence ([over.ics.scs])** are candidate functions. For direct-initialization, **those the permissible types for** explicit conversion functions **that are not hidden within S and yield type T or a type U that can be converted to type T with a (possibly trivial) qualification conversion ([conv.qual])** are also candidate functions; **otherwise there are none**. Conversion functions that return a *cv*-qualified type are considered to yield the *cv*-unqualified version of that type for this process of selecting candidate functions. A call to a conversion

function returning “reference to x ” is a glvalue of type x , and such a conversion function is therefore considered to yield x for this process of selecting candidate functions.

[over.match.ref]

Change paragraph 1:

- Under the conditions specified in [dcl.init.ref], a reference can be bound directly to the result of applying a conversion function to an initializer expression. Overload resolution is used to select the conversion function to be invoked. Assuming that “reference to $cv1\ T$ ” is the type of the reference being initialized, and “ $cv\ S$ ” is the type of the initializer expression, with S a class type, the candidate functions are selected as follows:

1. The conversion functions of S and its base classes are considered. Those non-explicit conversion functions that are not hidden within S and yield Let R be a set of types including “lvalue reference to $cv2\ T2$ ” (when initializing an lvalue reference or an rvalue reference to function) or “ $cv2\ T2$ ” or “rvalue reference to $cv2\ T2$ ” (when initializing an rvalue reference or an lvalue reference to function), for any $T2$. The permissible types for non-explicit conversion functions are the members of R where “ $cv1\ T$ ” is reference-compatible ([dcl.init.ref]) with “ $cv2\ T2$ ”, are candidate functions. For direct-initialization, those the permissible types for explicit conversion functions that are not hidden within S and yield type “lvalue reference to $cv2\ T2$ ” (when initializing an lvalue reference or an rvalue reference to function) or “rvalue reference to $cv2\ T2$ ” (when initializing an rvalue reference or an lvalue reference to function), the members of R where $T2$ is the same type as T or can be converted to type T with a (possibly trivial) qualification conversion ([conv.qual]), are also candidate functions; otherwise there are none.

[over.match.best]

Change paragraph 4:

- If the best viable function resolves to a function for which multiple declarations were found, and if at least any two of these declarations — or the declarations they refer to in the case of using declarations — specify a default argument for the same parameter that made the function viable, the program is ill-formed. [Example:

[...]

— end example]

[over.over]

Change paragraph 1:

- A use of a function name *id-expression* whose name refers to an overload set and that appears without arguments is resolved to a function, a pointer to function, or a pointer to member function for a specific function that is chosen from a set of selected functions determined based on the target type required in the context (if any), as described below. The target can be

1. an object or reference being initialized ([dcl.init], [dcl.init.ref], [dcl.init.list]),
2. the left side of an assignment ([expr.ass]),
3. a parameter of a function ([expr.call]),
4. a parameter of a user-defined operator ([over.oper]),
5. the return value of a function, operator function, or conversion ([stmt.return]),
6. an explicit type conversion ([expr.type.conv], [expr.static.cast], [expr.cast]), or
7. a non-type *template-parameter* ([temp.arg.nontype]).

The function name *id-expression* can be preceded by the & operator. [Note: Any redundant set of parentheses surrounding the function name is ignored ([expr.prim.paren]). — end note]

Change paragraph 3:

- For each function template designated by the name, The specialization, if any, generated by template argument deduction is done ([temp.over], [temp.deduct.funcaddr], [temp.arg.explicit]), and if the argument deduction succeeds, the resulting template argument list is used to generate a single for each function template specialization, which named is added to the set of selected functions considered. [Note: [...] — end note]

Change paragraph 4:

- Non-member functions and static member functions match targets of function pointer type or reference to function type. Non-static member functions match targets of pointer-to-member-function type. **[Note: If a non-static member function is selected/chosen, the reference to the overloaded function name is required to have the form of result can be used only to form a pointer to member as described in ([expr.unary.op]). — end note]**

[over.oper]

[over.unary]

Change paragraph 2:

- **[Note: The unary and binary forms of the same operator are considered to have the same name. [Note: Consequently, a unary operator can hide a binary operator from an enclosing scope, and vice versa. — end note]**

[over.literal]

Change paragraph 2:

- A declaration whose *declarator-id* is a *literal-operator-id* shall be a declaration of a namespace-scope function or function template **that belongs to a namespace** (it could be a friend function ([class.friend])), **or** an explicit instantiation or specialization of a function template, **or a using-declaration ([namespace.udecl]).** A function declared with a *literal-operator-id* is a *literal operator*. A function template declared with a *literal-operator-id* is a *literal operator template*.

[temp]

[temp.pre]

Change paragraph 4:

- A *template-declaration* can appear only as a namespace scope or class scope declaration. Its *declaration* shall not be an *export-declaration*. In a function template declaration, the last component of the *declarator-id* shall **not** be a **template-id name**. **[Note: That last component may be an identifier, an operator-function-id, a conversion-function-id, or a literal-operator-id.** In a class template declaration, if the **class name class-name** is a *simple-template-id*, the declaration declares a class template partial specialization ([temp.class.spec]). — end note]

Change paragraph 7:

- **[Note: A class-template shall not have the same name as any other template, class, function, variable, enumeration, enumerator, namespace, or type name bound in the same scope ([basic.scope]), except as specified in [temp.class.spec]. Except that a function template can be overloaded either by with non-template functions ([dcl.fct]) with the same name or by other and/or function templates with the same name ([temp.over]), a template name declared in namespace scope or in class scope shall be unique in that scope. Specializations, including partial specializations ([temp.class.spec]), do not bind names. — end note]**

[temp.param]

Change paragraph 15:

- A *template-parameter* shall not be given default arguments by two different declarations **in the same scope if one is reachable from the other.** [Example:

[...]

— end example]

[temp.names]

Remove the note in paragraph 1.

Change paragraph 2:

- For a *template-name* to be explicitly qualified by the template arguments, the name must be considered to refer to a template. [Note: Whether a name actually refers to a template cannot be known in some cases until after argument dependent lookup is done ([basic.lookup.argdep]). — end note] A name $\leq$ is considered to refer to interpreted as the delimiter of a *template-template-argument-list* if it follows a name, other than a *conversion-function-id*,
 1. that follows the keyword `template` after a *nested-name-specifier* or in a class member access expression, or
 2. for which name lookup finds the injected-class-name of a class template or finds a *template-name* or an overload set that contains a function template, by declaration of a template that, unless the name is in a closed context of a class or class template ([class.mem]), precedes the name, or
 3. A name is also considered to refer to a template if it that is an *unqualified-id* *unqualified name* followed by a $<$ and name lookup either finds one or more functions or finds nothing, or
 4. that is an *unqualified* or *terminal name* in a *using-declerator* ([namespace.udecl]), a *declarator-id* ([dcl.meaning]), or a *type-only context* ([temp.res]).

[Note: If the name is an *identifier*, it is then interpreted as a *template-name*. The keyword `template` is used to indicate that a qualified dependent name ([temp.dep.type]) denotes a template where an expression might appear or that a name in a class member access ([expr.ref]) is not dependent because it is part of a *nested-name-specifier*. — end note] If a *simple-template-id* results and is followed by a $::$, the name is looked up as part of a *nested-name-specifier*, discarding any previous lookup result.

Move the example from paragraph 4 to the end of paragraph 2.

Change paragraph 3:

- When a name is considered to be a *template-name*, and it is followed by a $\leq$, the $\leq$ is always taken as the delimiter of a *template-argument-list* and never as the less-than operator. When parsing a *template-argument-list*, the first non-nested $>[\dots]$ is taken as the ending delimiter rather than a greater-than operator. Similarly, the first non-nested $>>$ is treated as two consecutive but distinct $>$ tokens, the first of which is taken as the end of the *template-argument-list* and completes the *template-id*. [Note: The second $>$ token produced by this replacement rule may terminate an enclosing *template-id* construct or it may be part of a different construct (e.g., a cast). — end note] [Example:

[...]

— end example]

Change paragraph 4:

- The keyword `template` is said to A *nested-name-specifier* is *declarative* if it appears at the top level in a *qualified-id* if it appears outside of a *template-argument-list* or *decltype-specifier*. In a *qualified-id* of a *declarator-id* or in a *qualified-id* formed by a *class-head-name* ([class.pre]) or, *enum-head-name* ([dcl.enum]), for *declarative nested-name-specifier*. The keyword `template` shall not appear at the top level immediately after a *declarative nested-name-specifier*. In a *qualified-id* used as the name in a *typename-specifier* ([temp.res]), *elaborated-type-specifier* ([dcl.type.elab]), *using-declaration* ([namespace.udecl]), or *class-or-decltype* ([class.derived]), an optional keyword `template` appearing at the top level is ignored. In these contexts, a $\leq$ -token is always assumed to introduce a *template-argument-list*. In all other contexts, when naming a template specialization of a member of an unknown specialization ([temp.dep.type]), the member template name shall be prefixed by the keyword `template`. [Example:

[...]

— end example]

Change paragraph 5:

- A name prefixed by the keyword `template` shall be a *template-id* followed by a *template argument list* or the name shall refer to a class template or an alias template. The latter case is deprecated ([depr.template.template]). The keyword `template` shall not appear immediately before a $\sim$ -token (as to name a destructor). [Note: The keyword `template` may not be applied to non-template members of class templates. — end note][Note: As is the case with the *typename* prefix, the *template* prefix is allowed in cases where it is not strictly necessary; i.e., when the *nested name-specifier* or the expression on the left of the $\rightarrow$ or $\cdot$ is not dependent on a *template parameter*, or the use does not appear in the scope of even when lookup for the name would already find a template. — end note] [Example:

```
template <class T> struct A {  
    void f(int);
```



```

    template <class U> void f(U);
};

template <class T> void f(T t) {
    A<T> a;
    a.template f<>(t);           // OK: calls template
    a.template f(t);           // error: not a template-id
}

template <class T> struct B {
    template <class T2> struct C { };
};

// OK deprecated: T::template C would be assumed to name a class template:
template <class T, template <class X> class TT = T::template C> struct D { };
D<B<int> > db;

```

— end example]

Change paragraph 7:

- When the *template-name* of a *simple-template-id* names a constrained non-function template or a constrained template *template-parameter*, but not a member template that is a member of an unknown specialization ([temp.res]), and all *template-arguments* in the *simple-template-id* are non-dependent ([temp.dep.temp]), the associated constraints ([temp.constr.decl]) of the constrained template shall be satisfied ([temp.constr.constr]). [Example:

[...]

— end example]

[temp.arg]

Change paragraph 3:

- The names of used in a *template-argument* shall be accessible at the point where it is used as a *template-argument* are subject to access control where they appear. [Note: If the name of the *template-argument* is accessible at the point where it is used as a *template-argument*, there is no further access restriction in the resulting instantiation where the corresponding *template-parameter* name is used. — end note]

[...]

— end example] For a *template-argument* that is a class type or a class template, the template definition has no special access rights to the members of the *template-argument*. [Example:

[...]

— end example]

Change paragraph 7:

- When name lookup for the name in a *template-id* finds an overload set, both non-template functions in the overload set and function templates in the overload set for which the *template-arguments* do not match the *template-parameters* are ignored. [Note: If none of the function templates have matching *template-parameters*, the program is ill-formed. — end note]

[temp.arg.template]

Change paragraph 2:

- Any partial specializations ([temp.class.spec]) associated with the primary class template or primary variable template are considered when a specialization based on the template *template-parameter* is instantiated. If a specialization is not visible at reachable from the point of instantiation, and it would have been selected had it been visible, the program is ill-formed, no diagnostic required. [Example:

[...]

— end example]

[temp.decls]

[temp.mem]

Remove paragraph 6.

[temp.friend]

Change paragraph 1:

- A friend of a class or class template can be a function template or class template, a specialization of a function template or class template, or a non-template function or class. For a friend function declaration that is not a template declaration:
 1. if the ~~name of the friend~~ *unqualified-id* in the *declarator-id* is a ~~qualified or unqualified~~ *template-id*, the friend declaration refers to a specialization of a function template, otherwise,
 2. if the ~~name of the friend~~ *declarator-id* is a *qualified-id* and a matching non-template function is found in the specified class or namespace, the friend declaration refers to that function, otherwise,
 3. if the ~~name of the friend~~ *declarator-id* is a *qualified-id* and a matching function template is found in the specified class or namespace, the friend declaration refers to the deduced specialization of that function template ([temp.deduct.decl]), otherwise,
 4. the ~~name~~ *declarator-id* shall be an *unqualified-id* that declares (or redeclares) a non-template function.

[Example:

[...] — end example]

[temp.class.spec]

Change paragraph 1:

- A *primary class template* declaration is one in which the ~~class template name~~ *class-name* is an identifier. A template declaration in which the ~~class template name~~ *class-name* is a *simple-template-id* is a *partial specialization* of the class template named in the *simple-template-id*. A partial specialization of a class template provides an alternative definition of the template that is used instead of the primary definition when the arguments in a specialization match those given in the partial specialization ([temp.class.spec.match]). ~~A declaration of the primary template shall be declared before~~ *precede* any specializations of that template. A partial specialization shall ~~be declared before the first~~ *be reachable from any* use of a class template specialization that would make use of the partial specialization as the result of an implicit or explicit instantiation ~~in every translation unit in which such a use occurs~~; no diagnostic is required.

Change paragraph 7:

- Partial specialization declarations ~~themselves are not found by name lookup~~ *do not introduce a name*. ~~Rather~~ *Instead*, when the primary template name is used, any ~~previously-declared~~ *reachable* partial specializations of the primary template are also considered. One consequence is that a *using-declaration* which refers to a class template does not restrict the set of partial specializations which may be found through the *using-declaration*. [Example:

[...]

— end example]

Insert before paragraph 8:

- A specialization of a class template partial specialization that has been declared but not defined is an incomplete type ([basic.types]).

[temp.over.link]

Change paragraph 5:

- [...] For determining whether two dependent names ([temp.dep]) are equivalent, only the name itself is considered, not the result of name lookup in the context of the template. [Note: If multiple declarations of the same function template differ in the result of this such a dependent name lookup is unqualified, the result for it is looked up from the first declaration is used of the function template ([temp.dep.candidate]). — end note] [...]

[temp.res]

Replace paragraph 1:

- A name that appears in a template declaration is looked up from where it appears in the first declaration of the template that contains it. A *dependent name* is looked up for each specialization (after substitution) because the lookup depends on a template parameter, as described in [temp.dep]. [Note: Uses of other names might be type-dependent or value-dependent ([temp.dep.expr], [temp.dep.constexpr]). A *using-declarator* is never dependent in a specialization and is therefore replaced during lookup for a dependent name ([basic.lookup]). — end note] [Example:

```
struct A {operator int();};
template<class B, class T>
struct D : B {
    T get() {return operator T();}    // conversion-function-id is dependent
};
int f(D<A, int> d) {return d.get();} // OK: lookup finds A::operator int
```

— end example]

Move the example from paragraph 10 and the second example from [temp.dep]/4 to the end of paragraph 1.

Replace paragraph 2 (but not the grammar following it):

- If the validity or meaning of the program would be changed by considering a default argument or default template argument introduced in a declaration that is reachable from the point of instantiation of a specialization ([temp.point]) but is not found by lookup for the specialization, the program is ill-formed, no diagnostic required.

Change paragraph 3:

- A *typename-specifier* denotes the type or class template denoted by the *simple-type-specifier* ([dcl.type.simple]) formed by omitting the keyword *typename*. [Note: The usual qualified name lookup ([basic.lookup.qual]) is used to find the qualified-id applies even in the presence of *typename*. — end note] [Example:

[...]

— end example]

Replace paragraph 4:

- A name *appears as a type* in a construct if it is the construct or appears in it outside of any enclosed expression or template argument.

Change paragraph 5:

- A qualified-id qualified or unqualified name is assumed said to name be in a type only context if
 1. it appears as a type in a typename-specifier, nested-name-specifier, elaborated-type-specifier, or class-or-decltype.
 2. it appears as a type in a new-type-id, defining-type-id, or conversion-type-id.
 3. it is a qualified name in a type-id-only context (see below) appears as a type in a type-id that is a trailing-return-type, the default argument of a type-parameter, or the type-id of a static cast, const cast, reinterpret cast, or dynamic cast,
 4. it, or a template-id of which it is the name, is a *decl-specifier* of the *decl-specifier-seq* of a
 1. *simple-declaration* or a *function-definition* in namespace scope,
 2. *member-declaration*,
 3. *parameter-declaration* in a *member-declaration*[...], unless that *parameter-declaration* appears in a default argument,
 4. *parameter-declaration* in a *declarator* of a function or function template declaration whose *declarator-id* is qualified, unless that *parameter-declaration* appears in a default argument,

5. *parameter-declaration* in a *lambda-declarator*, unless that *parameter-declaration* appears in a default argument, or
6. *parameter-declaration* of a (non-type) *template-parameter*.

A qualified name is said to be in a *type-id-only context* if it appears in a *type-id*, *new-type-id*, or *defining-type-id* and the smallest enclosing *type-id*, *new-type-id*, or *defining-type-id* is a *new-type-id*, *defining-type-id*, *trailing-return-type*, default argument of a *type-parameter* of a template, or *type-id* of a *static_cast*, *const_cast*, *reinterpret_cast*, or *dynamic_cast*. [Example:

[...]

— end example]

Change paragraph 6:

- A *qualified-id* that refers to a member of an unknown specialization, that is not prefixed by *typename*, and *qualified dependent name* that is not otherwise assumed to name in a *type-only context* (see above) denotes is interpreted as a *non-qualified-id*, not a *type*. A name that refers to a dependent *using-declarator* is interpreted as a *typedef-name* if the *using-declarator* uses the keyword *typename* and as an *unqualified-id* otherwise. [Example:

[...]

— end example]

Remove paragraphs 7, 9, and 10 (whose example is moved to paragraph 1).

[temp.local]

Change paragraph 3:

- The injected-class-name of a class template or class template specialization can be used as either a *template-name* or a *type-name* wherever it is *in scope named*. [Example:

```
template <class T> struct Base {
    Base* p;
};

template <class T> struct Derived: public Base<T> {
    typename Derived::Base* p;           // meaning Derived::Base<T>
};

template<class T, template<class> class U = T::template Base> struct Third { };
Third<Derived<int> > t;                  // OK: default argument uses injected-class-name as a template
```

— end example]

Change paragraph 6:

- The name of a *template-parameter* shall not be *redeclared within its scope* (including nested scopes) *to which the template-parameter belongs*. A *template-parameter* shall not have the same name as the *template-name*. [Example:

[...]

— end example]

Change paragraph 7:

- In the definition of a member *Unqualified name lookup considers the template parameter scope of a template-declaration before any scope that contains the template declared (even if it does not contain the template-parameter-list)*. [Note: The scope of a class template that appears outside of the class template definition, the name of a member of the class template hides the name of a *template-parameter* of any enclosing class templates (but not a *template-parameter* of the member if the member is a class or function template), including its non-dependent base classes ([temp.dep.type], [class.member.lookup]), is searched before its template parameter scope. — end note] [Example:

```

struct B { };
namespace N {
    typedef void V;
    template<class T> struct A : B {
        struct B { /* ... */ };
        typedef void C;
        void f();
        template<class U> void g(U);
    };
}

template<class BV> void N::A<BV>::f() {
    BV bv; // A's B, not V is the template parameter, not N::V
}

template<class B> template<class C> void N::A<B>::g(C) {
    B b; // A's B the base class, not the template parameter
    C c; // the template parameter C, not A's C
}

```

— end example]

Remove paragraphs 8 and 9 (whose examples are merged into the above).

[temp.dep]

Change paragraph 2:

- The name in an *unqualified-id* is dependent if it is a *conversion-function-id* whose *conversion-type-id* is dependent or the *unqualified-id* is the *postfix-expression* in an expression of the form:

postfix-expression (*expression-list*_{opt})

where the *postfix-expression* is an *unqualified-id*, the *unqualified-id* denotes a dependent name if including when finding non-member candidates for an operator ([over.match.oper]), and

1. any of the expressions in the *expression-list* is a pack expansion ([temp.variadic]),
2. any of the expressions or *braced-init-lists* in the *expression-list* is type-dependent ([temp.dep.expr]), or
3. the *unqualified-id* is a *template-id* in which any of the template arguments depends on a template parameter.

If an operand of an operator is a type-dependent expression, the operator also denotes a dependent name. [Note: Such names are unbound and are looked up only at the point of the template instantiation ([temp.point]) in both the context of the template definition and the context of the point of instantiation ([temp.dep.candidate]). — end note]

Remove paragraph 4.

[temp.dep.type]

Change paragraph 1:

- A name *or template-id* refers to the *current instantiation* if it is

[...]

Change paragraph 5:

- A *qualified* ([basic.lookup.qual]) or *unqualified* name is a *member of the current instantiation* if it is
 1. its lookup context, if it is a qualified name, is the current instantiation, and
 2. An unqualified name that, when looked up, refers to at least one lookup for it (or its terminal name) finds any (perhaps inherited) member of a class that is the current instantiation or a non-dependent base class thereof. [Note: This can only occur when looking up a name in a scope enclosed by the definition of a class template. — end note]
 3. A *qualified-id* in which the *nested-name-specifier* refers to the current instantiation and that, when looked up, refers to at least one member of a class that is the current instantiation or a non-dependent base class thereof.

[Note: If no such member is found, and the current instantiation has any dependent base classes, then the *qualified-id* is a member of an unknown specialization; see below. — end note]

4. An *id-expression* denoting the member in a class member access expression ([*expr.ref*]) for which the type of the object expression is the current instantiation, and the *id-expression*, when looked up ([*basic.lookup.classref*]), refers to at least one member of a class that is the current instantiation or a non-dependent base class thereof. [Note: If no such member is found, and the current instantiation has any dependent base classes, then the *id-expression* is a member of an unknown specialization; see below. — end note]

[Example:

[...]

— end example]

A *qualified or unqualified* name is a *dependent member of the current instantiation* if it is a member of the current instantiation that, when looked up, refers to *denotes* at least one *direct* member of a class that is the current instantiation.

Change paragraph 6:

- A *qualified* name ([*basic.lookup.qual*]) is *a member of an unknown specialization* *dependent* if it is *not a member-qualified name that is a nested-name-specifier and*
 1. *its terminal name is a conversion-function-id whose conversion-type-id is dependent, or*
 2. *A qualified-id in which the nested-name-specifier names its lookup context is a dependent type that and is not the current instantiation, or*
 3. *A qualified-id in which the nested-name-specifier refers to its lookup context is the current instantiation, the current instantiation and has at least one dependent base class, and name lookup of for the terminal name of the qualified-id qualified name does not find any member of a class that is the current instantiation or a non-dependent base class thereof nothing ([basic.lookup.qual]).*
 4. [...]

A class member access expression is dependent if its id-expression is.

Change paragraph 7:

- If a *qualified-id in which the nested-name-specifier refers to qualified name (but not a member-qualified name that is a nested-name-specifier) whose lookup context is* the current instantiation is *not dependent and is* not a member of the current instantiation *or a member of an unknown specialization*, the program is ill-formed even if the template *whose definition* contains the *qualified-id qualified name* is not instantiated; no diagnostic required. *Similarly, if the id-expression in a class member access expression for which the type of the object expression is the current instantiation does not refer to a member of the current instantiation or a member of an unknown specialization, the program is ill-formed even if the template containing the member access expression is not instantiated; no diagnostic required.*

[Example:

```
template<class T> class A {
    typedef int type;
    void f() {
        A<T>::type i;           // OK: refers to a member of the current instantiation
        typename A<T>::other j; // error: neither a member of the current instantiation nor dependent
                                // a member of an unknown specialization
    }
};
```

— end example]

Change paragraph 8:

- If, for a given set of template arguments, a specialization of a template is instantiated that refers to a member of the current instantiation with a *qualified-id or class member access expression qualified name, the its terminal name in the qualified-id or class member access expression* is looked up in the template instantiation context. If the result of this lookup differs from the result of name lookup in the template definition context, name lookup is ambiguous. [Example:

[...]

— end example]

Change paragraph 9:

- A type is dependent if it is
 1. a template parameter,
 2. denoted by a member of an unknown specialization (qualified) dependent name,
 3. a nested class or enumeration that is a dependent direct member of a class that is the current instantiation,
 4. a cv-qualified type where the cv-unqualified type is dependent,
 5. a compound type constructed from any dependent type,
 6. an array type whose element type is dependent or whose bound (if any) is value-dependent,
 7. a function type whose exception specification is value-dependent,
 8. denoted by a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is a dependent type or an expression that is type-dependent or value-dependent or is a pack expansion [Note: This includes an injected-class-name ([class.pre]) of a class template used without a *template-argument-list*. — end note], or
 9. denoted by `decltype(expression)`, where *expression* is type-dependent ([temp.dep.expr]).

[temp.dep.expr]

Change paragraph 2:

- this is type-dependent if the current class type of the enclosing member function is dependent ([temp.dep.type]).

Change paragraph 3:

- An *id-expression* is type-dependent if it contains
 1. an identifier name associated by name lookup with one or more declarations declared with a dependent type,
 2. an identifier name associated by name lookup with a non-type *template-parameter* declared with a type that contains a placeholder type ([dcl.spec.auto]),
 3. an identifier name associated by name lookup with a variable declared with a type that contains a placeholder type ([dcl.spec.auto]) where the initializer is type-dependent,
 4. an identifier name associated by name lookup with one or more declarations of member functions of a class that is the current instantiation declared with a return type that contains a placeholder type,
 5. an identifier name associated by name lookup with a structured binding declaration ([dcl.struct.bind]) whose *brace-or-equal-initializer* is type-dependent,
 6. the identifier `__func__` ([dcl.fct.def.general]), where any enclosing function is a template, a member of a class template, or a generic lambda,
 7. a *template-id* that is dependent,
 8. a *conversion-function-id* that specifies a dependent type, or
 9. a *nested-name-specifier* or a *qualified-id* that names a member of an unknown specialization dependent name;

or if it names a dependent direct member of a class that is the current instantiation that is a static data member of type “array of unknown bound of τ ” for some τ ([temp.static]). Expressions of the following forms are type-dependent only if the type specified by the *type-id*, *simple-type-specifier* or *new-type-id* is dependent, even if any subexpression is type-dependent:

[...]

Change paragraph 5:

- A class member access expression ([expr.ref]) is type-dependent if the expression it is dependent or it refers to a member of the current instantiation and the type of the referenced member is dependent, or the class member access expression refers to a member of an unknown specialization. [Note: In an expression of the form `x.y` or `xp->y` the type of the expression is usually the type of the member `y` of the class of `x` (or the class pointed to by `xp`). However, if `x` or `xp` refers to a dependent type that is not the current instantiation, the type of `y` is always dependent. If `x` or `xp` refers to a non-dependent type or refers to the current instantiation, the type of `y` is the type of the class member access expression. — end note]

[temp.dep.temp]

Change paragraph 4:

- A template *template-argument* is dependent if it names a *template-parameter* or is a *qualified-id* that refers to a *member of an unknown specialization* *dependent name*.

[temp.nondep]

Remove subclause, referring the stable name to [temp.res] and removing the one cross reference (in [basic.def.odr]/12).

[temp.dep.candidate]

Change paragraph 1:

- For a function call where the *postfix-expression* is a dependent name, the candidate functions are found using the usual lookup rules from the template definition context ([basic.lookup.unqual], [basic.lookup.argdep]). [Note: For the part of the lookup using associated namespaces ([basic.lookup.argdep]), function declarations found in the template instantiation context are found by this lookup, as described in [basic.lookup.argdep]. — end note] If *a function* call whose *postfix-expression* is an *unqualified dependent name* would be ill-formed or would find a better match had the lookup *for the name* within the associated namespaces considered all the function declarations with external linkage introduced in those namespaces in all translation units, not just considering those declarations found in the template definition and template instantiation contexts ([basic.lookup.argdep]), then the program *has undefined behavior* *is ill-formed, no diagnostic required*.

[temp.inject]

Change paragraph 1:

- Friend classes *or, class templates*, functions, *or function templates* can be declared within a class template. When a template is instantiated, *the names of* its friends *declarations* are treated as if the specialization had been explicitly declared at its point of instantiation.

Remove paragraph 2.

[temp.spec]

Change paragraph 3:

- An explicit specialization may be declared for a function template, a variable template, a class template, a member of a class template, or a member template. An explicit specialization declaration is introduced by `template<>`. In an explicit specialization declaration for a variable template, a class template, a member of a class template or a class member template, *the name of* the variable or class that is explicitly specialized shall be *specified with* a *simple-template-id*. In the explicit specialization declaration for a function template or a member function template, *the name of* the function or member function explicitly specialized may be *specified using* a *template-id*. [Example:

[...]

— end example]

[temp.inst]

Change paragraph 17:

- [...] [Note: The satisfaction of constraints is *determined* *checked* during *name lookup* *or* overload resolution ([over.match.viable], [over.over]). — end note] [...]

[temp.explicit]

Change paragraph 4:

- If the explicit instantiation is for a class or member class, the *elaborated-type-specifier* in the *declaration* shall include a *simple-template-id*; otherwise, the *declaration* shall be a *simple-declaration* whose *init-declarator-list* comprises a single *init-declarator* that does not have an *initializer*. *If the explicit instantiation is for a function or member function, the unqualified-id in the declarator shall be either a template-id or, where all template arguments can be deduced, a*

~~template-name or operator-function-id. [Note: The declaration may declare a *qualified-id*, in which case the *unqualified-id* of the *qualified-id* must be a *template-id*. — end note]~~ If the explicit instantiation is for a member function, a member class or a static data member of a class template specialization, ~~the name of~~ the class template specialization **shall be named** in the *qualified-id* ~~for the member name shall be with~~ a *simple-template-id*. If the explicit instantiation is for a variable template specialization, the *unqualified-id* in the *declarator* shall be a *simple-template-id*. ~~An explicit instantiation shall appear in an enclosing namespace of its template. If the name declared in the explicit instantiation is an unqualified name, the explicit instantiation shall appear in the namespace where its template is declared or, if that namespace is inline ([namespace.def]), any namespace from its enclosing namespace set. [Note: Regarding qualified names in declarators, see [decl.meaning]. — end note]~~ [Example:

[...]

— end example]

Change paragraph 5:

- **An explicit instantiation does not introduce a name ([basic.scope.scope]).** A declaration of a function template, a variable template, a member function or static data member of a class template, or a member function template of a class or class template shall **precede be reachable from** any explicit instantiation of that entity. A definition of a class template, a member class of a class template, or a member class template of a class or class template shall **precede be reachable from** any explicit instantiation of that entity unless ~~the explicit instantiation is preceded by~~ an explicit specialization of the entity with the same template arguments **is reachable therefrom**. If the *declaration* of the explicit instantiation names an implicitly-declared special member function ([special]), the program is ill-formed.

Remove paragraph 8.

Change paragraph 9:

- A trailing *template-argument* can be left unspecified in an explicit instantiation of a function template specialization or of a member function template specialization provided it can be deduced from the type of a function parameter ([temp.deduct]). **If all template arguments can be deduced, the empty template argument list <> may be omitted.** [Example:

[...]

— end example]

[temp.expl.spec]

Change paragraph 4:

- **An explicit specialization does not introduce a name ([basic.scope.scope]).** A declaration of a function template, class template, or variable template being explicitly specialized shall **precede be reachable from** the declaration of the explicit specialization. [Note: A declaration, but not a definition of the template is required. — end note] The definition of a class or class template shall **precede be reachable from** the declaration of an explicit specialization for a member template of the class or class template. [Example:

[...]

— end example]

Change paragraph 5:

- A member function, a member function template, a member class, a member enumeration, a member class template, a static data member, or a static data member template of a class template may be explicitly specialized for a class specialization that is implicitly instantiated; in this case, the definition of the class template shall **precede be reachable from** the explicit specialization for the member of the class template. If such an explicit specialization for the member of a class template names an implicitly-declared special member function ([special]), the program is ill-formed.

Remove paragraph 9.

[temp.fct.spec]

[temp.arg.explicit]

Change paragraph 4:

- Trailing template arguments that can be deduced ([temp.deduct]) or obtained from default *template-arguments* may be omitted from the list of explicit *template-arguments*. **Note:** A trailing template parameter pack ([temp.variadic]) not otherwise deduced will be deduced as an empty sequence of template arguments. **— end note** If all of the template arguments can be deduced, they may all be omitted; in this case, the empty template argument list <> itself may also be omitted. **In contexts where deduction is done and fails, or in contexts where deduction is not done, if a template argument list is specified and it, along with any default template arguments, identifies a single function template specialization, then the *template-id* is an lvalue for the function template specialization.** [Example:

[...]

— end example]

[temp.deduct]

Change example in bullet (11.4):

- [Example:

```
template <int I> struct X { };
template <template <class T> class> struct Z { };
template <class T> void f(typename T::Y*){}
template <class T> void g(X<T::N>*){}
template <class T> void h(Z<T::template TT>*){}
struct A {};
struct B { int Y; };
struct C {
    typedef int N;
};
struct D {
    typedef int TT;
};

int main() {
    // Deduction fails in each of these cases:
    f<A>(0);           // A does not contain a member Y
    f<B>(0);           // The Y member of B is not a type
    g<C>(0);           // The N member of C is not a non-type
    h<D>(0);           // The TT member of D is not a template
}
```

— end example]

[temp.deduct.conv]

Change paragraph 1:

- Template argument deduction is done by comparing the return type of the conversion function template (call it P) with the type **that is required as the result specified by the *conversion-type-id* of the conversion *conversion-function-id* (call it A; see [del.init], [over.match.conv], and [over.match.ref] for the determination of that type) as described in [temp.deduct.type].** **If the *conversion-function-id* is constructed during overload resolution ([over.match.funcs]), the following transformations apply.**

Change paragraph 5:

- In general, the deduction process attempts to find template argument values that will make the deduced A identical to A. However, **there are four cases that allow a difference certain attributes of A may be ignored:**
 1. If the original A is a reference type, **any cv-qualifiers of A can be more cv-qualified than the deduced A** (i.e., the type referred to by the reference).
 2. If the original A is a function pointer **or pointer-to-member-function** type, **A can be “pointer to function” even if the deduced A is “pointer to its noexcept function”.**
 3. **If the original A is a pointer-to-member-function type, A can be “pointer to member of type function” even if the deduced A is “pointer to member of type noexcept function”.**
 4. **The deduced Any cv-qualifiers in A can be another pointer or pointer-to-member type that can be converted to A via restored by a qualification conversion.**

Change paragraph 6:

- These alternatives are considered ignored only if type deduction would otherwise fail. If they yield ignoring them allows more than one possible deduced A, the type deduction fails.

[temp.over]

Change paragraph 1:

- When a call to the name of a function or function template is written (explicitly, or implicitly using the operator notation), template argument deduction ([temp.deduct]) and checking of any explicit template arguments ([temp.arg]) are performed for each function template to find the template argument values (if any) that can be used with that function template to instantiate a function template specialization that can be invoked with the call arguments. For each function template, if the argument deduction and checking succeeds, the *template-arguments* (deduced and/or explicit) are used to synthesize the declaration of a single function template specialization which is added to the candidate functions set to be used in overload resolution. If, for a given function template, argument deduction fails or the synthesized function template specialization would be ill-formed, no such function is added to the set of candidate functions for that template. The complete set of candidate functions includes all the synthesized declarations and all of the non-template overloaded functions of the same name found by name lookup. The synthesized declarations are treated like any other functions in the remainder of overload resolution, except as explicitly noted in [over.match.best]. [...]

[except.handle]

Remove paragraph 11.

[diff.basic]

Change paragraph 2 (the example is based on that from the removed [class.nested.type]):

- **Affected subclause:** [basic.scope]
Change: A struct is a scope in C++, not in C.
For example,

```
struct X {  
    struct Y { int a; } b;  
};  
struct Y c;
```

is valid in C but not in C++, which would require `X::Y c;`.

Rationale: Class scope is crucial to C++, and a struct is a class.

Effect on original feature: Change to semantics of well-defined feature.

Difficulty of converting: Semantic transformation.

How widely used: C programs use struct extremely frequently, but the change is only noticeable when struct, enumeration, or enumerator names are referred to outside the struct. The latter is probably rare.

[depr.template.template]

Add this subclause before [depr.c.headers]:

- The use of the keyword `template` before the qualified name of a class or alias template without a template argument list is deprecated.

Acknowledgments

Thanks to Richard Smith for helping discover and interpret several of the issues addressed and for feedback on their resolutions, and to Rachel Ertl for helping decide on them.